

Flow Matching et Modèles de Diffusion

Fondements mathématiques, transport optimal et mise en pratique

Félix Bos

16 juillet 2026

Table des matières

1	Objectif du cours	3
2	Cadre probabiliste et notations	3
2.1	Espace de travail	3
2.2	Variables aléatoires, réalisations et convention d'indices	4
3	Génération comme transport de mesures	4
3.1	Flot déterministe et théorème de Cauchy–Lipschitz	4
3.2	Dérivation heuristique par bilan de flux	5
3.3	Équation de continuité (formulation forte)	6
3.4	Formulation faible	6
4	Transport optimal et formule de Benamou–Brenier	7
4.1	Problème de Monge–Kantorovich	7
4.2	Formulation dynamique de Benamou–Brenier	7
4.3	Théorème de Brenier et caractérisation du transport optimal quadratique	8
5	Méthode classique : les <i>Continuous Normalizing Flows</i>	8
5.1	Modèle et vraisemblance exacte	8
5.2	Formule de changement de variables instantanée	9
5.3	Coût de l'entraînement par maximum de vraisemblance	9
6	Construction des chemins de probabilité conditionnels	10
6.1	Couplage entre source et cible	10
6.2	Chemin de probabilité conditionnel	10
6.3	Vitesse conditionnelle	11
6.4	Champ de vitesse marginal et théorème de marginalisation	11
7	Schedules d'interpolation gaussienne	12
7.1	Schedule linéaire (rectified flow)	12
7.2	Schedule cosinus	12
7.3	Cadre gaussien général : schedules <i>variance preserving</i> et <i>variance exploding</i>	13
7.4	Analyse de covariance	13
8	Objectif d'apprentissage : flow matching	13
8.1	Risque quadratique conditionnel	13
8.2	Théorème de régression et équivalence des minimiseurs	14

9	Transport optimal et flow matching	15
9.1	Couplages par transport optimal en minibatch (OT-CFM)	15
9.2	Optimal Flow Matching : un flot rectiligne exact en une étape (OFM)	16
10	Génération par intégration numérique	17
10.1	ODE de génération et erreur de discrétisation	17
11	Exemple 1 : distribution 2D make_moons	18
11.1	Construction du dataset	18
11.2	Code minimal	18
11.3	Schéma du modèle et sortie du notebook	20
11.4	Variante OT-CFM sur make_moons : comparaison des vecteurs	20
12	Exemple 2 : MNIST avec U-Net	20
12.1	Passage aux images	20
12.2	Interpolation utilisée	21
12.3	Boucle d'entraînement	21
12.4	Schéma du U-Net et sortie du notebook	22
12.5	Variante OT-CFM sur MNIST : comparaison des vecteurs en espace image	23
13	Comparaison expérimentale : CFM contre OT-CFM	23
13.1	Rectitude des trajectoires et énergie cinétique sur make_moons	25
13.2	Qualité de génération en fonction du nombre de pas (NFE)	25
13.3	Sur MNIST : qualité visuelle à faible NFE	27
14	Résumé conceptuel	27
15	Exercices	29
16	Ouverture	30

1 Objectif du cours

Le *flow matching* est une méthode de modélisation générative dont le but est d'apprendre à transformer progressivement une distribution simple, comme un bruit gaussien, en une distribution beaucoup plus complexe représentant des données réelles. Dans le cas de la génération d'images, le modèle part donc d'un échantillon aléatoire sans structure apparente et apprend à le faire évoluer jusqu'à obtenir une image plausible.

L'idée fondamentale est de ne pas chercher à produire directement une donnée en une seule étape, mais de décrire la génération comme une transformation continue entre deux états : un état initial correspondant au bruit et un état final correspondant aux données. Cette transformation est représentée par un champ de vitesse dépendant du temps, c'est-à-dire une application qui indique, à chaque instant et en chaque point de l'espace, la direction et l'intensité du déplacement local de la masse de probabilité.

Cette approche permet d'interpréter la génération comme un problème de *transport de mesures* : la masse probabiliste associée au bruit est progressivement déplacée, réorganisée et déformée jusqu'à reproduire la structure de la distribution des données. Cette vision rapproche naturellement le flow matching de la théorie du transport optimal (Monge–Kantorovich), qui cherche également à transformer une mesure en une autre tout en contrôlant le « coût » de cette transformation.

Dans ce cours, on cherchera à établir rigoureusement, dans un ordre délibérément progressif – des fondations analytiques vers les raffinements les plus récents :

- le cadre probabiliste et analytique permettant de définir un flot continu de mesures de probabilité (équation de continuité, formulation faible) ;
- le lien entre ce flot et le transport optimal (formule de Benamou–Brenier, théorème de Brenier), qui donne un critère précis d'« optimalité » d'une trajectoire ;
- la méthode *classique*, historiquement antérieure, des *Continuous Normalizing Flows* entraînés par maximum de vraisemblance exacte, et les limites pratiques (coût de simulation, adjoint) qui ont motivé l'introduction du flow matching ;
- la construction de chemins de probabilité conditionnels et le passage du champ conditionnel au champ marginal par un argument de régression et de désintégration de mesure – c'est-à-dire le flow matching proprement dit, entraîné sans aucune simulation d'ODE ;
- le retour au transport optimal pour *améliorer* le flow matching : couplages par transport optimal en minibatch (OT-CFM), puis flot rectiligne exact obtenu par paramétrisation convexe du champ de vitesse (Optimal Flow Matching) ;
- les schémas d'intégration numérique de l'ODE apprise et leur ordre de convergence ;
- la mise en pratique complète sur `make_moons` (2D) puis sur MNIST à l'aide d'une architecture U-Net (`notebook.ipynb`) ;
- une comparaison expérimentale directe entre un modèle CFM et un modèle OT-CFM entraînés de bout en bout sur les mêmes données (`ot_flow_matching.ipynb`, Section 13) : rectitude des trajectoires, énergie cinétique, et qualité de génération en fonction du nombre de pas d'intégration.

L'objectif n'est donc pas seulement d'implémenter un modèle génératif, mais de comprendre, dans un même fil conducteur, pourquoi la méthode classique (CNF) a été supplantée par le flow matching, et pourquoi le flow matching lui-même continue d'être perfectionné à l'aide de la théorie du transport optimal.

2 Cadre probabiliste et notations

2.1 Espace de travail

On se place sur un espace de probabilité $(\Omega, \mathcal{F}, \mathbb{P})$ suffisamment riche pour porter toutes les variables aléatoires considérées. On fixe un entier $d \geq 1$ et l'on travaille dans l'espace euclidien

$(\mathbb{R}^d, \langle \cdot, \cdot \rangle)$ muni de la tribu borélienne $\mathcal{B}(\mathbb{R}^d)$. Pour une image MNIST de taille $1 \times 28 \times 28$, on prend $d = 784$ après vectorisation.

On note :

- $\|\cdot\|_2$ la norme euclidienne sur $\mathbb{R}^d$;
- $\langle \cdot, \cdot \rangle$ le produit scalaire euclidien ;
- $\nabla \cdot f = \sum_{i=1}^d \partial_{x_i} f_i$ la divergence d'un champ $f : \mathbb{R}^d \rightarrow \mathbb{R}^d$ de classe $\mathcal{C}^1$;
- $\Delta\varphi = \nabla \cdot (\nabla\varphi)$ le laplacien d'une fonction scalaire φ ;
- $\text{Law}(X)$ la loi de la variable aléatoire X , c'est-à-dire la mesure image $X_{\#}\mathbb{P}$;
- $\mathcal{P}(\mathbb{R}^d)$ l'ensemble des mesures de probabilité boréliennes sur $\mathbb{R}^d$, et $\mathcal{P}_2(\mathbb{R}^d) \subset \mathcal{P}(\mathbb{R}^d)$ le sous-ensemble des mesures de moment d'ordre deux fini ;
- $\mathbb{E}[\cdot]$ l'espérance, $\text{Cov}(X) = \mathbb{E}[(X - \mathbb{E}[X])(X - \mathbb{E}[X])^\top]$ la matrice de covariance de X .

La distribution source est notée $p_0 \in \mathcal{P}_2(\mathbb{R}^d)$. Dans ce cours, on prendra le plus souvent

$$p_0 = \mathcal{N}(0, I_d),$$

où I_d désigne la matrice identité de taille d . La distribution cible, c'est-à-dire la distribution des données, est notée p_1 ou $p_{\text{data}} \in \mathcal{P}_2(\mathbb{R}^d)$. On n'en connaît généralement pas la densité explicite ; on possède seulement des échantillons i.i.d. $x_1^{(1)}, \dots, x_1^{(n)} \sim p_{\text{data}}$.

Définition 2.1 (Modèle génératif). Un modèle génératif est une procédure, éventuellement aléatoire, qui produit une variable aléatoire $\hat{X}_1$ dont la loi $\text{Law}(\hat{X}_1)$ doit approximer la distribution cible p_{data} , au sens d'une divergence statistique $D(\text{Law}(\hat{X}_1) \| p_{\text{data}})$ (par exemple une divergence de Kullback–Leibler KL ou une distance de Wasserstein W_2).

2.2 Variables aléatoires, réalisations et convention d'indices

On utilisera la convention suivante, standard dans la littérature du flow matching :

- les lettres majuscules X_0, X_1, X_t désignent des variables aléatoires ;
- les lettres minuscules x_0, x_1, x_t désignent des réalisations déterministes ;
- $t \in [0, 1]$ est un temps continu, $T \sim \mathcal{U}([0, 1])$ lorsqu'il est aléatoire ;
- pour tout t , X_t est une variable aléatoire de $\mathbb{R}^d$, et $p_t = \text{Law}(X_t)$ sa loi, appelée *marginale au temps t* .

Ainsi, écrire $X_0 \sim p_0$ signifie que la variable aléatoire X_0 suit la loi p_0 , tandis que x_0 est une valeur particulière tirée selon cette loi. On notera $\stackrel{\text{p.s.}}{=}$ l'égalité presque sûre et $\stackrel{\text{p.p.}}{=}$ l'égalité presque partout (Lebesgue).

Hypothèse 2.1 (Régularité). Sauf mention contraire, on suppose que pour tout $t \in [0, 1]$ la loi p_t admet une densité (encore notée p_t) par rapport à la mesure de Lebesgue, que cette densité est strictement positive et de classe $\mathcal{C}^1$ en (t, x) , et que le champ de vitesse v_t considéré est localement lipschitzien en x , uniformément en t , avec croissance au plus linéaire. Ces conditions garantissent l'existence et l'unicité globale des solutions de l'ODE (théorème de Cauchy–Lipschitz) ainsi que la validité des intégrations par parties utilisées plus bas.

3 Génération comme transport de mesures

3.1 Flot déterministe et théorème de Cauchy–Lipschitz

Soit un champ de vitesse dépendant du temps :

$$v : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d, \quad (t, x) \mapsto v_t(x).$$

Pour une condition initiale $x_0 \in \mathbb{R}^d$, on définit la trajectoire $t \mapsto x_t$ comme solution du problème de Cauchy :

$$\frac{dx_t}{dt} = v_t(x_t), \quad x_{t=0} = x_0. \quad (\text{ODE})$$

Théorème 3.1 (Cauchy–Lipschitz, existence et unicité). *Sous l’Hypothèse 2.1, pour tout $x_0 \in \mathbb{R}^d$ il existe une unique solution $t \mapsto x_t \in \mathcal{C}^1([0, 1], \mathbb{R}^d)$ de (ODE). De plus, l’application $x_0 \mapsto x_t$, notée Φ_t , est un $\mathcal{C}^1$ -difféomorphisme de $\mathbb{R}^d$ dans lui-même, appelé flot associé à v .*

On admet ce résultat classique d’analyse (voir par exemple Cauchy–Lipschitz–Picard). Il justifie que l’on puisse parler du flot Φ_t sans ambiguïté dès lors que v est suffisamment régulier.

Définition 3.1 (Pushforward). Soit $\mu \in \mathcal{P}(\mathbb{R}^d)$ et $F : \mathbb{R}^d \rightarrow \mathbb{R}^d$ une application mesurable. Le pushforward (mesure image) de μ par F , noté $F_{\#}\mu$, est l’unique mesure de probabilité telle que

$$\forall A \in \mathcal{B}(\mathbb{R}^d), \quad (F_{\#}\mu)(A) = \mu(F^{-1}(A)).$$

De manière équivalente, pour toute fonction test φ borélienne bornée,

$$\int_{\mathbb{R}^d} \varphi(y) d(F_{\#}\mu)(y) = \int_{\mathbb{R}^d} \varphi(F(x)) d\mu(x).$$

Si $X_0 \sim p_0$ et $X_t = \Phi_t(X_0)$, alors, par définition du pushforward,

$$p_t = (\Phi_t)_{\#}p_0.$$

L’objectif génératif du flow matching est donc de construire un champ v_t tel que

$$p_{t=1} = (\Phi_1)_{\#}p_0 \approx p_{\text{data}}.$$

3.2 Dérivation heuristique par bilan de flux

Avant d’énoncer (EC) sous sa forme condensée, il est utile de comprendre d’où elle vient par un argument physique élémentaire, le même qui gouverne la conservation de la matière en mécanique des fluides ou l’écoulement de véhicules sur une route.

Plaçons-nous en dimension $d = 1$ pour fixer les idées. La quantité $p_t(x)$ joue le rôle d’une densité de probabilité au point x à l’instant t (comme une densité de véhicules par kilomètre), et $v_t(x)$ celui d’une vitesse locale (comme la vitesse des véhicules en ce point). Le produit

$$\text{flux}(t, x) := p_t(x) v_t(x)$$

représente alors la quantité de masse probabiliste qui traverse le point x par unité de temps, dans le sens des x croissants – exactement comme « densité $\times$ vitesse » donne un débit de véhicules par heure.

Considérons un petit intervalle $[x, x + dx]$. La masse de probabilité qu’il contient à l’instant t vaut $p_t(x) dx$. Comme aucune masse n’apparaît ni ne disparaît (la probabilité totale reste égale à 1 à chaque instant), la seule façon pour cette masse de varier est qu’il en entre par la gauche et qu’il en sorte par la droite :

$$\underbrace{\partial_t p_t(x) dx}_{\text{variation de la masse dans } [x, x+dx]} \approx \underbrace{p_t(x) v_t(x)}_{\text{flux entrant en } x} - \underbrace{p_t(x+dx) v_t(x+dx)}_{\text{flux sortant en } x+dx}.$$

En divisant par dx et en faisant tendre $dx \rightarrow 0$, le membre de droite devient, par définition de la dérivée, $-\partial_x(p_t(x)v_t(x))$. On obtient ainsi

$$\partial_t p_t(x) = -\partial_x(p_t(x)v_t(x)),$$

qui est exactement (EC) en dimension 1. En dimension d quelconque, le même bilan appliqué à un petit volume V autour de x , combiné au théorème de flux-divergence (le flux net sortant à travers le bord ∂V vaut $\int_{\partial V} p_t v_t \cdot n \, dS \approx \nabla \cdot (p_t v_t)(x) |V|$ lorsque V est petit), donne de la même façon $\partial_t p_t + \nabla \cdot (p_t v_t) = 0$. Autrement dit, **(EC) n'est rien d'autre qu'un bilan de masse local** : la probabilité ne peut varier en un point que par ce qui y entre ou en sort, et ce qui y entre ou en sort est entièrement dicté par le champ v_t .

Exemple 3.1 (Vérification explicite sur un cas résoluble). Prenons $d = 1$, $v_t(x) = x$ et $p_0 = \mathcal{N}(0, 1)$. En résolvant (ODE), $\Phi_t(x_0) = x_0 e^t$, si bien que $X_t = X_0 e^t \sim \mathcal{N}(0, \sigma(t)^2)$ avec $\sigma(t) = e^t$, et

$$p_t(x) = \frac{1}{\sigma(t)\sqrt{2\pi}} \exp\left(-\frac{x^2}{2\sigma(t)^2}\right).$$

Calculons séparément les deux termes de (EC). Comme $\dot{\sigma} = \sigma$,

$$\partial_t p_t(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2\sigma^2} \left(-\frac{1}{\sigma} + \frac{x^2}{\sigma^3}\right),$$

et, avec $v_t(x) = x$,

$$\partial_x(x p_t(x)) = p_t(x) + x \partial_x p_t(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2\sigma^2} \left(\frac{1}{\sigma} - \frac{x^2}{\sigma^3}\right).$$

La somme des deux expressions est identiquement nulle : (EC) est donc vérifiée ici par un calcul direct et explicite, sans invoquer aucun argument abstrait.

Remarque 3.1. Cet exemple ne fonctionne que parce que p_t possède une formule close. Dans la quasi-totalité des cas pratiques (un champ v_t paramétré par un réseau de neurones), p_t n'est jamais connue explicitement, et une vérification directe comme celle de l'Exemple 3.1 est impossible. C'est précisément pour cette raison que la Section 3.3 introduit une formulation *faible* de (EC), qui ne nécessite de manipuler que des espérances – calculables même sans connaître p_t – plutôt que la densité elle-même.

3.3 Équation de continuité (formulation forte)

Proposition 3.1 (Équation de continuité). *Sous l'Hypothèse 2.1, la famille $(p_t)_{t \in [0,1]}$ définie par $p_t = (\Phi_t)_\# p_0$ vérifie, au sens classique,*

$$\boxed{\partial_t p_t(x) + \nabla \cdot (p_t(x) v_t(x)) = 0, \quad (t, x) \in [0, 1] \times \mathbb{R}^d.} \quad (\text{EC})$$

Cette équation, dite *équation de continuité* (ou équation de Liouville dans le contexte hamiltonien), formalise le fait que la masse probabiliste se déplace sous l'effet du champ v_t sans être créée ni détruite : c'est l'analogue probabiliste de la conservation de la masse en mécanique des fluides.

3.4 Formulation faible

Il est souvent plus commode, notamment lorsque p_t n'est pas régulière, de travailler avec la formulation faible de (EC), au sens des distributions.

Définition 3.2 (Solution faible). On dit que $(p_t)_{t \in [0,1]} \subset \mathcal{P}(\mathbb{R}^d)$ est solution faible de (EC) associée au champ v si, pour toute fonction test $\varphi \in \mathcal{C}_c^\infty(\mathbb{R}^d)$,

$$\frac{d}{dt} \int_{\mathbb{R}^d} \varphi(x) \, dp_t(x) = \int_{\mathbb{R}^d} \langle \nabla \varphi(x), v_t(x) \rangle \, dp_t(x). \quad (\text{EC-faible})$$

Proposition 3.2 (Équivalence probabiliste). *Soit $X_t = \Phi_t(X_0)$ avec $X_0 \sim p_0$. Alors, pour toute $\varphi \in \mathcal{C}_c^\infty(\mathbb{R}^d)$,*

$$\frac{d}{dt} \mathbb{E}[\varphi(X_t)] = \mathbb{E}[\langle \nabla \varphi(X_t), v_t(X_t) \rangle].$$

En particulier, (EC-faible) est équivalente à l'énoncé selon lequel $t \mapsto \mathbb{E}[\varphi(X_t)]$ est dérivable de dérivée $\mathbb{E}[\langle \nabla \varphi(X_t), v_t(X_t) \rangle]$ pour toute fonction test φ .

Démonstration. Par la formule de dérivation de fonctions composées appliquée à $t \mapsto \varphi(\Phi_t(x_0))$, on a, pour tout x_0 ,

$$\frac{d}{dt} \varphi(\Phi_t(x_0)) = \langle \nabla \varphi(\Phi_t(x_0)), \partial_t \Phi_t(x_0) \rangle = \langle \nabla \varphi(\Phi_t(x_0)), v_t(\Phi_t(x_0)) \rangle,$$

en utilisant (ODE). En prenant l'espérance sur $X_0 \sim p_0$ et en permutant espérance et dérivée en t (licite sous l'Hypothèse 2.1 par domination locale), on obtient le résultat annoncé, qui est exactement (EC-faible) écrite en termes de l'espérance $\int \varphi dp_t = \mathbb{E}[\varphi(X_t)]$. $\square$

Démonstration alternative. En partant de $\int \varphi(x) p_t(x) dx = \mathbb{E}[\varphi(X_t)]$ et de la Proposition 3.2,

$$\frac{d}{dt} \int_{\mathbb{R}^d} \varphi(x) p_t(x) dx = \int_{\mathbb{R}^d} \langle \nabla \varphi(x), v_t(x) \rangle p_t(x) dx.$$

Le membre de gauche vaut $\int \varphi(x) \partial_t p_t(x) dx$ par dérivation sous le signe intégral. Le membre de droite s'intègre par parties : comme φ est à support compact, les termes de bord s'annulent et

$$\int_{\mathbb{R}^d} \langle \nabla \varphi(x), v_t(x) \rangle p_t(x) dx = - \int_{\mathbb{R}^d} \varphi(x) \nabla \cdot (p_t(x) v_t(x)) dx.$$

En égalant les deux expressions pour toute $\varphi \in \mathcal{C}_c^\infty(\mathbb{R}^d)$, on conclut par le lemme fondamental du calcul des variations que $\partial_t p_t = -\nabla \cdot (p_t v_t)$ presque partout. $\square$

4 Transport optimal et formule de Benamou–Brenier

Cette section relie le flow matching à la théorie du transport optimal, ce qui éclaire pourquoi certains couplages sont préférables à d'autres.

4.1 Problème de Monge–Kantorovich

Définition 4.1 (Couplage). Un couplage entre $p_0, p_1 \in \mathcal{P}(\mathbb{R}^d)$ est une mesure de probabilité π sur $\mathbb{R}^d \times \mathbb{R}^d$ dont les marginales sont p_0 et p_1 :

$$\pi \in \Pi(p_0, p_1) := \left\{ \pi \in \mathcal{P}(\mathbb{R}^d \times \mathbb{R}^d) : (\text{proj}_1)_\# \pi = p_0, (\text{proj}_2)_\# \pi = p_1 \right\}.$$

Définition 4.2 (Distance de Wasserstein W_2). Pour $p_0, p_1 \in \mathcal{P}_2(\mathbb{R}^d)$, on définit

$$W_2(p_0, p_1)^2 := \inf_{\pi \in \Pi(p_0, p_1)} \int_{\mathbb{R}^d \times \mathbb{R}^d} \|x_0 - x_1\|_2^2 d\pi(x_0, x_1) = \inf_{\substack{(X_0, X_1) \\ \text{Law}(X_0)=p_0, \text{Law}(X_1)=p_1}} \mathbb{E}[\|X_0 - X_1\|_2^2].$$

4.2 Formulation dynamique de Benamou–Brenier

Théorème 4.1 (Benamou–Brenier, 2000). *Pour $p_0, p_1 \in \mathcal{P}_2(\mathbb{R}^d)$,*

$$W_2(p_0, p_1)^2 = \inf_{(p_t, v_t)} \left\{ \int_0^1 \int_{\mathbb{R}^d} \|v_t(x)\|_2^2 p_t(x) dx dt : \partial_t p_t + \nabla \cdot (p_t v_t) = 0, p_{t=0} = p_0, p_{t=1} = p_1 \right\}.$$

Le minimiseur est atteint par des trajectoires géodésiques $X_t = (1-t)X_0 + tX_1$ où (X_0, X_1) suit le couplage optimal de Monge (sous les hypothèses du théorème de Brenier).

Remarque 4.1. Ce théorème donne un sens précis à l'intuition selon laquelle le *couplage optimal* minimise l'énergie cinétique moyenne du transport. Le flow matching, tel que présenté aux sections suivantes, n'impose pas en général un couplage optimal : le couplage indépendant $\pi = p_0 \otimes p_1$ est généralement utilisé par simplicité, au prix d'une énergie cinétique plus élevée (et donc de trajectoires plus courbes) que le couplage de Monge. La Section 9 montrera comment se rapprocher du régime optimal du Théorème 4.1, d'abord de façon approchée par des couplages de transport optimal en minibatch, puis de façon exacte grâce à une paramétrisation convexe du champ de vitesse.

4.3 Théorème de Brenier et caractérisation du transport optimal quadratique

Le Théorème 4.1 caractérise le *coût* du transport optimal. Le théorème suivant, dû à Brenier (1991), caractérise la *forme* de l'application de transport optimal elle-même, pour le coût quadratique. Il sera central dans la Section 9.

Théorème 4.2 (Brenier, 1991). *Soit $p_0, p_1 \in \mathcal{P}_2(\mathbb{R}^d)$ avec p_0 absolument continue par rapport à la mesure de Lebesgue. Il existe une unique (à constante additive près) fonction convexe $\varphi^* : \mathbb{R}^d \rightarrow \mathbb{R}$ telle que $T^* = \nabla \varphi^*$ pousse p_0 sur p_1 :*

$$T_{\#}^* p_0 = p_1,$$

et T^* est, p_0 -presque partout, l'unique solution du problème de Monge

$$T^* \in \arg \min_{T_{\#} p_0 = p_1} \int_{\mathbb{R}^d} \|x - T(x)\|_2^2 dp_0(x).$$

En particulier, $(\text{Id}, T^*)_{\#} p_0$ est l'unique couplage optimal $\pi^* \in \Pi(p_0, p_1)$ atteignant l'infimum définissant $W_2(p_0, p_1)^2$.

Remarque 4.2. On admet ce résultat classique de transport optimal (voir Villani, 2003, ou Santambrogio, 2015, pour une démonstration complète reposant sur la dualité de Kantorovich). Il implique en particulier que les trajectoires géodésiques du Théorème 4.1, $X_t = (1-t)X_0 + tT^*(X_0)$, sont associées à une vitesse conditionnelle *constante en t* : $U_t = T^*(X_0) - X_0$. C'est précisément cette structure que la Section 9 cherchera à exploiter pour apprendre des trajectoires rectilignes.

5 Méthode classique : les *Continuous Normalizing Flows*

Avant d'introduire le flow matching à proprement parler, il est instructif de présenter la méthode historiquement antérieure qui utilise le *même* objet mathématique – un flot Φ_t transportant p_0 vers p_1 – mais avec un critère d'entraînement radicalement différent : le maximum de vraisemblance exacte. Cette méthode, dite des *Continuous Normalizing Flows* (CNF, Chen *et al.*, 2018), permet de comprendre précisément quel problème pratique le flow matching est venu résoudre.

5.1 Modèle et vraisemblance exacte

On reprend le cadre de la Section 3 : un champ $v_\theta : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ paramétré par un réseau de neurones engendre un flot $\Phi_{\theta,t}$, avec $p_{\theta,0} = p_0$ connue explicitement (typiquement $\mathcal{N}(0, I_d)$) et

$$p_{\theta,1} = (\Phi_{\theta,1})_{\#} p_0.$$

Contrairement au flow matching, où l'on ne calcule jamais explicitement $p_{\theta,1}$, l'objectif d'un CNF est de maximiser la vraisemblance exacte des données sous ce modèle :

$$\max_{\theta} \mathbb{E}_{X_1 \sim p_{\text{data}}} [\log p_{\theta,1}(X_1)].$$

Comme $\Phi_{\theta,t}$ est un difféomorphisme (Théorème de Cauchy–Lipschitz), le calcul de $p_{\theta,1}(x_1)$ passe par le *flot inverse* $x_0 = \Phi_{\theta,1}^{-1}(x_1) = \Phi_{\theta,0}(x_1)$, obtenu en intégrant l'ODE à rebours du temps $t = 1$ à $t = 0$.

5.2 Formule de changement de variables instantanée

Proposition 5.1 (Changement de variables instantané, Chen *et al.*, 2018). *Sous l’Hypothèse 2.1, le long de toute trajectoire $x_t = \Phi_t(x_0)$,*

$$\boxed{\frac{d}{dt} \log p_t(x_t) = -\nabla \cdot v_t(x_t) = -\text{Tr}(\nabla_x v_t(x_t)).}$$

Par intégration entre 0 et 1,

$$\log p_1(x_1) = \log p_0(x_0) - \int_0^1 \text{Tr}(\nabla_x v_t(x_t)) dt.$$

Démonstration. Posons $\ell(t) = \log p_t(x_t)$. Par dérivation composée,

$$\dot{\ell}(t) = \frac{\partial_t p_t(x_t)}{p_t(x_t)} + \langle \nabla \log p_t(x_t), \dot{x}_t \rangle = \frac{\partial_t p_t(x_t)}{p_t(x_t)} + \langle \nabla \log p_t(x_t), v_t(x_t) \rangle,$$

en utilisant (ODE). Par l’équation de continuité (EC) établie à la Proposition 3.1,

$$\partial_t p_t(x) = -\nabla \cdot (p_t v_t)(x) = -p_t(x) \text{Tr}(\nabla v_t(x)) - \langle \nabla p_t(x), v_t(x) \rangle,$$

d’où, en $x = x_t$ et en divisant par $p_t(x_t)$,

$$\frac{\partial_t p_t(x_t)}{p_t(x_t)} = -\text{Tr}(\nabla v_t(x_t)) - \langle \nabla \log p_t(x_t), v_t(x_t) \rangle.$$

En substituant dans l’expression de $\dot{\ell}(t)$, les deux termes en $\langle \nabla \log p_t, v_t \rangle$ s’annulent exactement, ce qui laisse $\dot{\ell}(t) = -\text{Tr}(\nabla v_t(x_t))$. Le résultat s’obtient en intégrant cette identité entre $t = 0$ et $t = 1$. $\square$

Remarque 5.1. Cette formule remplace élégamment le déterminant jacobien de la formule usuelle de changement de variables ($p_1(x_1) = p_0(x_0) |\det \nabla \Phi_1(x_0)|^{-1}$) par une simple intégrale de trace le long de la trajectoire, ce qui évite tout calcul de déterminant en dimension d (coût $O(d^3)$).

5.3 Coût de l’entraînement par maximum de vraisemblance

La perte de maximum de vraisemblance négative pour un point de donnée x_1 s’écrit, d’après la Proposition 5.1,

$$-\log p_{\theta,1}(x_1) = -\log p_0(\Phi_{\theta,0}(x_1)) + \int_0^1 \text{Tr}(\nabla_x v_{\theta,t}(x_t)) dt.$$

Évaluer cette perte, *et* son gradient par rapport à θ , nécessite en pratique :

1. de résoudre numériquement l’ODE à rebours pour obtenir x_0 à partir de x_1 , et simultanément d’intégrer le terme de trace le long de cette trajectoire (un solveur ODE adaptatif à pas variable est utilisé, dont le nombre d’évaluations du réseau n’est pas fixé à l’avance) ;
2. d’évaluer, à chaque pas, la trace du jacobien $\nabla_x v_{\theta,t}$, de coût exact $O(d)$ passes arrière (ou approchée en $O(1)$ passe par l’estimateur stochastique de Hutchinson, $\text{Tr}(\nabla v_t(x)) \approx \varepsilon^\top \nabla v_t(x) \varepsilon$ pour $\varepsilon \sim \mathcal{N}(0, I_d)$, comme dans FFFJORD, Grathwohl *et al.*, 2018) ;
3. de rétropropager à travers l’ensemble de la trajectoire d’intégration, soit en gardant en mémoire tous les états intermédiaires (coût mémoire proportionnel au nombre de pas), soit par la *méthode de sensibilité adjointe*.

Proposition 5.2 (Méthode adjointe, Chen *et al.*, 2018, d’après Pontryagin). *Soit L une perte scalaire dépendant de $x_1 = \Phi_{\theta,1}(x_0)$. L’état adjoint $a(t) := (\partial L / \partial x_t)^\top$ vérifie l’ODE rétrograde*

$$\frac{da(t)}{dt} = -a(t)^\top \nabla_{x_t} v_{\theta,t}(x_t), \quad a(1) = \frac{\partial L}{\partial x_1},$$

et le gradient par rapport aux paramètres s’obtient par

$$\frac{\partial L}{\partial \theta} = - \int_0^1 a(t)^\top \frac{\partial v_{\theta,t}}{\partial \theta}(x_t) dt.$$

Remarque 5.2. La méthode adjointe permet de calculer le gradient exact avec un coût mémoire $O(1)$, indépendant du nombre de pas d’intégration, au prix d’une seconde résolution d’ODE (rétrograde, couplée) à chaque passe d’entraînement.

Point d’attention

Limites motivant le flow matching. L’entraînement d’un CNF par maximum de vraisemblance est dit *simulation-based* : chaque exemple d’entraînement nécessite de résoudre au moins une ODE (souvent deux, avec la méthode adjointe), et d’estimer une trace de jacobien. Le coût est variable, potentiellement élevé, et croît avec la complexité du champ appris. De plus, rien dans ce critère d’entraînement ne favorise des trajectoires simples ou rectilignes : le Théorème 4.1 n’est pas exploité. Le flow matching (Section 6 et suivantes) contourne entièrement ce problème : la perte de régression $\mathcal{L}_{\text{CFM}}$ ne requiert **aucune intégration d’ODE pendant l’entraînement**, uniquement une évaluation directe du réseau en un point (t, x_t) tiré par simulation *a priori* du chemin conditionnel, dont la vitesse cible U_t est calculable en forme close. C’est ce gain – « simulation-free training » – qui a rendu le flow matching praticable à grande échelle.

6 Construction des chemins de probabilité conditionnels

6.1 Couplage entre source et cible

Le flow matching ne manipule pas seulement les deux lois marginales p_0 et p_1 mais un couplage $\pi \in \Pi(p_0, p_1)$ entre elles. Le couplage le plus simple, utilisé par défaut, est le couplage indépendant $\pi = p_0 \otimes p_1$ (auquel cas $X_0 \perp\!\!\!\perp X_1$). Des couplages plus sophistiqués (transport optimal exact ou approché) réduisent l’énergie cinétique du chemin et donc la courbure des trajectoires apprises, mais ne modifient pas la théorie qui suit.

6.2 Chemin de probabilité conditionnel

On se donne, en plus du couplage π , une variable de conditionnement $Z = (X_0, X_1) \sim \pi$ (on pourrait aussi conditionner uniquement sur X_1 ; voir Remarque 6.1).

Définition 6.1 (Chemin conditionnel affine). Soient $\alpha, \beta : [0, 1] \rightarrow \mathbb{R}$ de classe $\mathcal{C}^1$ vérifiant les conditions aux bords

$$\alpha(0) = 1, \beta(0) = 0, \quad \alpha(1) = 0, \beta(1) = 1.$$

Pour $z = (x_0, x_1) \in \mathbb{R}^d \times \mathbb{R}^d$, on définit le chemin conditionnel

$$\psi_t(z) = \alpha(t)x_0 + \beta(t)x_1,$$

et, pour $Z = (X_0, X_1) \sim \pi$, la variable intermédiaire

$$X_t := \psi_t(Z) = \alpha(t)X_0 + \beta(t)X_1.$$

Par construction, $X_{t=0} = X_0$ (p.s.) et $X_{t=1} = X_1$ (p.s.). La *loi conditionnelle* au temps t sachant $Z = z$ est la masse de Dirac $p_t(\cdot | z) = \delta_{\psi_t(z)}$, et la *marginale* au temps t s'obtient par désintégration :

$$p_t(x) = \int_{\mathbb{R}^d \times \mathbb{R}^d} p_t(x | z) \, d\pi(z) = (\psi_t)_\# \pi(x).$$

Remarque 6.1. Dans la littérature (Lipman *et al.*, 2023), on conditionne souvent seulement sur $Z = X_1$ (avec $X_0 \sim p_0$ tiré indépendamment à chaque fois), ce qui revient exactement au cas du couplage indépendant $\pi = p_0 \otimes p_1$ présenté ici. Toute la théorie qui suit reste valable en remplaçant $Z = (X_0, X_1)$ par $Z = X_1$ seul.

6.3 Vitesse conditionnelle

Définition 6.2 (Vitesse conditionnelle). La vitesse conditionnelle associée au chemin ψ_t est

$$u_t(x | z) := \partial_t \psi_t(z) = \dot{\alpha}(t)x_0 + \dot{\beta}(t)x_1, \quad z = (x_0, x_1).$$

Au niveau des variables aléatoires, on note $U_t := u_t(X_t | Z) = \dot{\alpha}(t)X_0 + \dot{\beta}(t)X_1$.

La variable U_t est la vitesse de la trajectoire conditionnelle : c'est la vitesse que l'on connaîtrait si la paire complète $Z = (X_0, X_1)$ était observée. Elle vérifie trivialement, pour π -presque tout z , l'équation de continuité conditionnelle

$$\partial_t p_t(x | z) + \nabla \cdot (p_t(x | z) u_t(x | z)) = 0 \quad (\text{EC-cond})$$

au sens des distributions (c'est l'équation de transport d'une masse de Dirac le long de la trajectoire $\psi_t(z)$).

6.4 Champ de vitesse marginal et théorème de marginalisation

Pendant la génération, le modèle ne connaît pas la paire cachée Z : il ne connaît que la position courante $X_t = x$ et le temps t . Le champ que l'on doit effectivement apprendre est donc une espérance conditionnelle.

Définition 6.3 (Champ marginal). Le champ de vitesse marginal associé au couplage π est

$$v_t^\pi(x) := \mathbb{E}[U_t | X_t = x] = \int_{\mathbb{R}^d \times \mathbb{R}^d} u_t(x | z) \, d\pi(z | X_t = x),$$

où $\pi(\cdot | X_t = x)$ est la loi conditionnelle (désintégration) de Z sachant $X_t = x$, définie p_t -presque partout par le théorème de désintégration de Rokhlin.

Théorème 6.1 (Théorème de marginalisation, Lipman *et al.*, 2023). *Sous l'Hypothèse 2.1, si pour π -presque tout z le couple $(p_t(\cdot | z), u_t(\cdot | z))$ vérifie (EC-cond), alors la marginale $p_t = \int p_t(\cdot | z) \, d\pi(z)$ vérifie l'équation de continuité (EC) avec le champ marginal v_t^π défini ci-dessus :*

$$\partial_t p_t(x) + \nabla \cdot (p_t(x) v_t^\pi(x)) = 0.$$

Autrement dit, le champ moyen conditionnel transporte exactement la marginale, alors même que v_t^π ne dépend que de (t, x) et non de la variable cachée z .

Démonstration. Fixons $\varphi \in \mathcal{C}_c^\infty(\mathbb{R}^d)$. Par la formule des espérances totales (Fubini, licite sous l'Hypothèse 2.1),

$$\frac{d}{dt} \int_{\mathbb{R}^d} \varphi(x) p_t(x) \, dx = \frac{d}{dt} \int_{\mathbb{R}^d \times \mathbb{R}^d} \varphi(\psi_t(z)) \, d\pi(z) = \int_{\mathbb{R}^d \times \mathbb{R}^d} \langle \nabla \varphi(\psi_t(z)), u_t(\psi_t(z) | z) \rangle \, d\pi(z),$$

en dérivant sous le signe intégral (domination assurée par l'Hypothèse 2.1) et en utilisant $\partial_t \psi_t(z) = u_t(\psi_t(z) \mid z)$. On réécrit l'intégrale double par désintégration $d\pi(z) = d\pi(z \mid x) dp_t(x)$ (théorème de Rokhlin, avec $x = \psi_t(z)$) :

$$\int_{\mathbb{R}^d \times \mathbb{R}^d} \langle \nabla \varphi(\psi_t(z)), u_t(\psi_t(z) \mid z) \rangle d\pi(z) = \int_{\mathbb{R}^d} \left\langle \nabla \varphi(x), \underbrace{\int u_t(x \mid z) d\pi(z \mid x)}_{= v_t^\pi(x)} \right\rangle p_t(x) dx.$$

On a donc établi

$$\frac{d}{dt} \int_{\mathbb{R}^d} \varphi(x) p_t(x) dx = \int_{\mathbb{R}^d} \langle \nabla \varphi(x), v_t^\pi(x) \rangle p_t(x) dx,$$

qui est exactement (EC-faible) pour le couple (p_t, v_t^π) . Le Théorème 6.1 s'en déduit par l'équivalence entre formulation faible et formulation forte (Proposition 3.1). $\square$

Intuition

Chaque paire (X_0, X_1) fournit une trajectoire déterministe et donc une vitesse exacte U_t . Mais au moment de générer, on ne connaît pas la paire cachée : plusieurs trajectoires conditionnelles distinctes peuvent passer par le même point x au même instant t . Le champ marginal $v_t^\pi(x)$ est la moyenne de ces vitesses conditionnelles, pondérée par la probabilité *a posteriori* de chaque trajectoire sachant $X_t = x$. Le Théorème 6.1 garantit que suivre cette vitesse moyenne suffit à transporter correctement la loi marginale, sans jamais avoir besoin de connaître individuellement les trajectoires conditionnelles.

7 Schedules d'interpolation gaussienne

7.1 Schedule linéaire (rectified flow)

Le schedule linéaire est défini par $\alpha(t) = 1 - t$, $\beta(t) = t$, de sorte que

$$X_t = (1 - t)X_0 + tX_1, \quad \dot{\alpha}(t) = -1, \quad \dot{\beta}(t) = 1, \quad U_t = X_1 - X_0.$$

La vitesse conditionnelle ne dépend pas explicitement du temps : les trajectoires conditionnelles sont des segments de droite parcourus à vitesse constante. C'est le chemin utilisé par le *rectified flow* (Liu *et al.*, 2022).

7.2 Schedule cosinus

Le schedule cosinus (*trigonometric interpolant*) est défini par

$$\alpha(t) = \cos\left(\frac{\pi}{2}t\right), \quad \beta(t) = \sin\left(\frac{\pi}{2}t\right),$$

qui vérifie bien $\alpha(0) = 1, \beta(0) = 0, \alpha(1) = 0, \beta(1) = 1$. Les dérivées temporelles sont

$$\dot{\alpha}(t) = -\frac{\pi}{2} \sin\left(\frac{\pi}{2}t\right), \quad \dot{\beta}(t) = \frac{\pi}{2} \cos\left(\frac{\pi}{2}t\right),$$

d'où la vitesse conditionnelle

$$U_t = -\frac{\pi}{2} \sin\left(\frac{\pi}{2}t\right) X_0 + \frac{\pi}{2} \cos\left(\frac{\pi}{2}t\right) X_1.$$

7.3 Cadre gaussien général : schedules *variance preserving* et *variance exploding*

Plus généralement, on peut écrire tout chemin gaussien conditionnel sous la forme

$$X_t = \alpha(t)X_1 + \sigma(t)\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I_d) \text{ indépendant de } X_1,$$

ce qui correspond, en identifiant $X_0 := \varepsilon \sim p_0 = \mathcal{N}(0, I_d)$ et en échangeant les rôles de α, β ci-dessus, au cas particulier $\beta \equiv \alpha(\cdot)$ appliqué à X_1 et $\alpha \equiv \sigma$ (attention à l'inversion de notation avec la sous-section précédente : ici α pondère la donnée X_1 et σ pondère le bruit). Deux familles se distinguent selon la contrainte imposée sur $\alpha(t)^2 + \sigma(t)^2$:

Schedule	Définition	Propriété
Linéaire (rectified flow)	$\alpha(t) = t, \sigma(t) = 1 - t$	énergie cinétique minimale sur droites, mais covariance non constante
Cosinus	$\alpha(t) = \sin(\frac{\pi}{2}t), \sigma(t) = \cos(\frac{\pi}{2}t)$	$\alpha(t)^2 + \sigma(t)^2 \equiv 1$: <i>variance preserving</i> (VP)
<i>Variance Exploding</i> (VE)	$\alpha(t) \equiv 1, \sigma(t)$ croissant de $\sigma_{\min}$ à $\sigma_{\max} \gg 1$	covariance $\Sigma_1 + \sigma(t)^2 I_d$ non bornée ; utilisé par Song–Ermon (SMLD)

7.4 Analyse de covariance

On suppose ici $X_0 \perp\!\!\!\perp X_1$, $\text{Cov}(X_0) = \Sigma_0$, $\text{Cov}(X_1) = \Sigma_1$, et l'on revient à la paramétrisation $X_t = \alpha(t)X_0 + \beta(t)X_1$ du §7.1–7.2. Par indépendance, le terme de covariance croisée est nul et

$$\text{Cov}(X_t) = \alpha(t)^2 \Sigma_0 + \beta(t)^2 \Sigma_1.$$

Proposition 7.1. *Si $\Sigma_0 = \Sigma_1 = I_d$, alors $\text{Cov}(X_t) = (\alpha(t)^2 + \beta(t)^2)I_d$. Pour le schedule linéaire, $\alpha(t)^2 + \beta(t)^2 = (1-t)^2 + t^2$, qui atteint son minimum $1/2$ en $t = 1/2$: la loi intermédiaire est artificiellement contractée. Pour le schedule cosinus,*

$$\alpha(t)^2 + \beta(t)^2 = \cos^2\left(\frac{\pi}{2}t\right) + \sin^2\left(\frac{\pi}{2}t\right) = 1 \quad \forall t \in [0, 1],$$

si bien que $\text{Cov}(X_t) = I_d$ pour tout t : le chemin est *variance preserving* au sens strict.

Point d'attention

La propriété « variance preserving » n'est garantie ici que dans le cas idéalisé où $X_0 \perp\!\!\!\perp X_1$ et $\Sigma_0 = \Sigma_1 = I_d$. Pour une distribution cible p_{data} de covariance quelconque $\Sigma_1 \neq I_d$, la covariance de X_t dépend de t même avec le schedule cosinus ; la terminologie renvoie à un régime de référence, pas à une propriété universelle.

8 Objectif d'apprentissage : flow matching

8.1 Risque quadratique conditionnel

Soit $T \sim \mathcal{U}([0, 1])$ indépendant de (X_0, X_1) . On construit $X_T = \psi_T(X_0, X_1)$ et $U_T = u_T(X_T | (X_0, X_1))$. On paramètre un champ de vitesse par un réseau de neurones

$$v_\theta : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d, \quad (t, x) \mapsto v_\theta(t, x), \quad \theta \in \mathbb{R}^p.$$

Définition 8.1 (Perte de flow matching conditionnel).

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{T, X_0, X_1} \left[\|v_\theta(T, X_T) - U_T\|_2^2 \right].$$

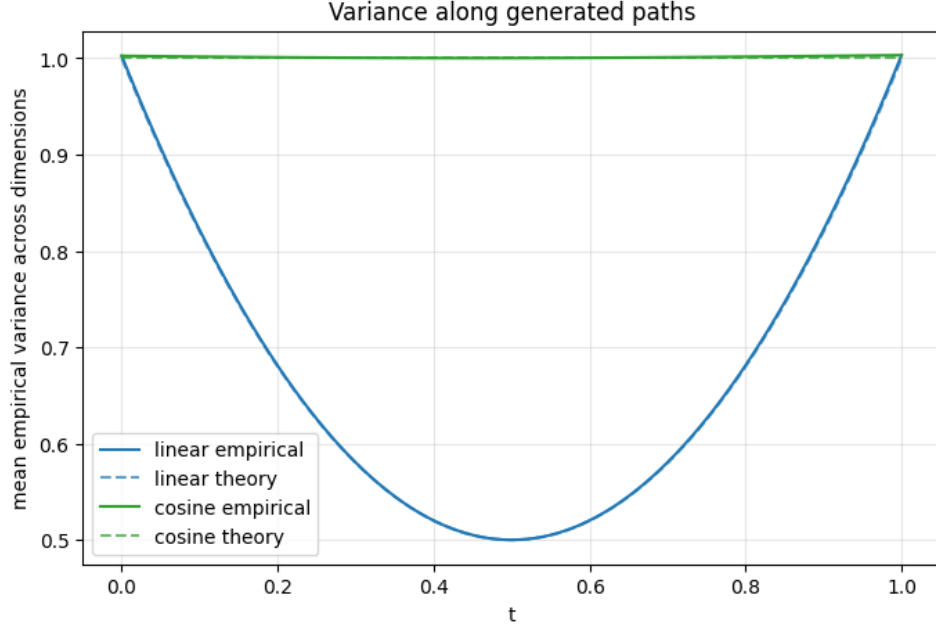


FIGURE 1 – Figure extraite du notebook : variance empirique et variance théorique le long des chemins linéaire et cosinus. Le schedule linéaire contracte la variance au milieu du transport, tandis que le schedule cosinus conserve l'échelle dans le cas isotrope.

8.2 Théorème de régression et équivalence des minimiseurs

Théorème 8.1 (Minimiseur de la perte quadratique). *Parmi toutes les fonctions mesurables $v : [0, 1] \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ telles que $\mathbb{E}[\|v(T, X_T)\|_2^2] < \infty$, le minimiseur du risque*

$$\mathcal{L}(v) = \mathbb{E} \left[\|v(T, X_T) - U_T\|_2^2 \right]$$

est, à un ensemble de mesure nulle près pour la loi de (T, X_T) ,

$$v^*(t, x) = \mathbb{E}[U_t \mid X_t = x] = v_t^\pi(x).$$

Démonstration. C'est la propriété classique de meilleure approximation en moyenne quadratique. Pour une variable aléatoire $Y \in L^2$ et une tribu $\sigma(Z)$, on a, pour toute fonction mesurable g telle que $g(Z) \in L^2$,

$$\mathbb{E}[\|g(Z) - Y\|_2^2] = \mathbb{E}[\|g(Z) - \mathbb{E}[Y \mid Z]\|_2^2] + \mathbb{E}[\|Y - \mathbb{E}[Y \mid Z]\|_2^2],$$

par le théorème de Pythagore dans $L^2(\Omega, \sigma(Z), \mathbb{P})$ (le terme croisé s'annule car $g(Z) - \mathbb{E}[Y \mid Z]$ et $Y - \mathbb{E}[Y \mid Z]$ sont orthogonaux : l'un est $\sigma(Z)$ -mesurable, l'autre est centré conditionnellement à Z). Le second terme ne dépend pas de g : le minimum en g est donc atteint (et uniquement atteint, $p_T \otimes \text{Law}(X_T)$ -presque partout) pour $g(Z) = \mathbb{E}[Y \mid Z]$. On applique ceci à $Z = (T, X_T)$ et $Y = U_T$. $\square$

Corollaire 8.1. *En combinant le Théorème 8.1 et le Théorème 6.1, le minimiseur théorique $v_\theta^* = v_t^\pi$ de $\mathcal{L}_{\text{CFM}}$ est exactement le champ qui transporte p_0 vers p_1 au sens de l'équation de continuité. Minimiser une perte de régression simple sur des vitesses conditionnelles, faciles à calculer en forme close, suffit donc à apprendre le champ marginal qui résout le problème de transport.*

Remarque 8.1 (Variance du gradient stochastique). La perte $\mathcal{L}_{\text{CFM}}$ a le même minimiseur que la perte « intractable » $\mathcal{L}(\theta) = \mathbb{E}_T[\int \|v_\theta(T, x) - v_T^\pi(x)\|_2^2 p_T(x) dx]$, mais son gradient stochastique a

une variance différente : le choix du couplage π (indépendant vs. transport optimal en minibatch) influence donc la variance de l'estimateur de gradient et la vitesse de convergence de l'entraînement, bien que le minimiseur θ^* reste, en théorie, inchangé pour un réseau suffisamment expressif. C'est précisément ce levier que la section suivante exploite.

9 Transport optimal et flow matching

Le Corollaire du Théorème 8.1 et le Théorème 6.1 montrent que **n'importe quel** couplage $\pi \in \Pi(p_0, p_1)$ donne, à la limite de l'entraînement, un champ marginal v_t^π qui transporte correctement p_0 vers p_1 . Mais la Remarque 4.1 a souligné que tous les couplages ne se valent pas : le couplage indépendant $\pi = p_0 \otimes p_1$, utilisé par défaut aux Sections 6 à 8, ne minimise pas l'énergie cinétique du Théorème 4.1, ce qui se traduit en pratique par des trajectoires courbes et un plus grand nombre de pas d'intégration nécessaires à la génération (Section 10). Cette section présente deux méthodes, de rigueur croissante, pour rapprocher le flow matching du régime optimal caractérisé par les Théorèmes 4.1 et 4.2.

9.1 Couplages par transport optimal en minibatch (OT-CFM)

L'idée de Tong *et al.* (2023, *Improving and generalizing flow-based generative models with minibatch optimal transport*, arXiv :2302.00482) est de remplacer, à chaque étape d'entraînement, l'échantillonnage indépendant de (X_0, X_1) par un échantillonnage selon une approximation empirique du couplage optimal, calculée à l'échelle d'un minibatch.

Définition 9.1 (Couplage optimal en minibatch). Soient $x_0^{(1)}, \dots, x_0^{(m)} \stackrel{\text{i.i.d.}}{\sim} p_0$ et $x_1^{(1)}, \dots, x_1^{(m)} \stackrel{\text{i.i.d.}}{\sim} p_1$ un minibatch. On résout le problème de transport optimal discret

$$\hat{P} \in \arg \min_{P \in \mathcal{U}_m} \sum_{i,j=1}^m P_{ij} \|x_0^{(i)} - x_1^{(j)}\|_2^2, \quad \mathcal{U}_m = \left\{ P \in \mathbb{R}_+^{m \times m} : P\mathbf{1} = \frac{1}{m}\mathbf{1}, P^\top \mathbf{1} = \frac{1}{m}\mathbf{1} \right\},$$

par l'algorithme hongrois (coût exact, $O(m^3)$) ou par régularisation entropique et l'algorithme de Sinkhorn (approché, $O(m^2)$ par itération). La paire (X_0, X_1) est alors tirée selon la loi discrète de matrice $\hat{P}$ plutôt que selon le produit $p_0 \otimes p_1$.

On applique ensuite exactement la même perte de flow matching conditionnel $\mathcal{L}_{\text{CFM}}$ que dans la Section 8, en tirant les paires (X_0, X_1) selon $\hat{P}$ plutôt qu'indépendamment. Le Théorème 6.1 et le Théorème 8.1 s'appliquent sans modification : quel que soit le couplage utilisé pour tirer (X_0, X_1) , la perte reste valide et son minimiseur reste un champ transportant correctement p_0 vers p_1 . Seule la *variance du gradient* et la *forme des trajectoires apprises* changent (Remarque 8.1).

Proposition 9.1 (Convergence vers le transport optimal dynamique, Tong *et al.*, 2023). *Lorsque le couplage optimal continu $\pi^* = (\text{Id}, T^*)_{\#} p_0$ du Théorème 4.2 existe, le couplage optimal en minibatch $\hat{P}$ en constitue une approximation empirique, dont Tong et al. (2023) montrent qu'elle converge, lorsque la taille de minibatch $m \rightarrow \infty$, vers le plan de transport optimal dynamique du Théorème 4.1. Le champ appris avec ce couplage (dit OT-CFM) approche donc, à taille de minibatch suffisante, les trajectoires d'énergie cinétique minimale.*

Remarque 9.1. Le surcoût de calcul du couplage optimal en minibatch (résolution d'un problème de transport de taille $m \times m$, avec m la taille du minibatch, typiquement quelques centaines) est négligeable devant le coût d'un pas de descente de gradient sur un grand réseau, et n'intervient qu'à l'entraînement : contrairement au CNF (Section 5), **aucune intégration d'ODE** n'est requise, ni à l'entraînement ni à l'inférence. Seule la procédure d'échantillonnage des paires (X_0, X_1) change. En pratique, des trajectoires moins courbes réduisent le nombre de pas d'intégration nécessaires lors de la génération (Section 10), sans en changer la nature.

9.2 Optimal Flow Matching : un flot rectiligne exact en une étape (OFM)

Le couplage OT-CFM ci-dessus n'est qu'une *approximation*, de qualité croissante avec la taille de minibatch mais jamais exacte à taille finie, et sujette au bruit d'échantillonnage. Kornilov, Mokrov, Gasnikov et Korotin (2024, *Optimal Flow Matching : Learning Straight Trajectories in Just One Step*, arXiv :2403.13117) proposent une méthode exploitant directement le Théorème 4.2 pour retrouver, en une seule étape d'entraînement (sans procédure itérative de « reflow » ni heuristique de minibatch), le couplage de Monge exact pour le coût quadratique.

Définition 9.2 (Paramétrisation convexe du champ de vitesse). On paramètre non pas directement v_θ , mais un potentiel $\varphi_\theta : \mathbb{R}^d \rightarrow \mathbb{R}$ contraint à être convexe – par exemple à l'aide d'un réseau à convexité garantie par construction (*Input Convex Neural Network*, Amos et al., 2017) – et l'on pose

$$T_\theta := \nabla \varphi_\theta, \quad X_t := (1-t)X_0 + tT_\theta(X_0), \quad v_\theta(t, X_t) := T_\theta(X_0) - X_0.$$

Le chemin est donc, par construction, le chemin linéaire du § 7.1 appliqué non pas à X_1 directement, mais à son estimation $T_\theta(X_0)$ par le potentiel appris.

Théorème 9.1 (Optimal Flow Matching, Kornilov et al., 2024). *Sous des hypothèses de régularité et d'expressivité suffisante de la classe des potentiels convexes $\{\varphi_\theta\}$, le minimiseur de la perte de flow matching conditionnel restreinte à cette paramétrisation,*

$$\theta^* \in \arg \min_{\theta} \mathbb{E}_{T, X_0, X_1} \left[\| (T_\theta(X_0) - X_0) - U_T \|_2^2 \right],$$

vérifie $T_{\theta^} = T^*$ (p_0 -presque partout), où $T^* = \nabla \varphi^*$ est l'application de transport optimal du Théorème 4.2. Le flot appris est donc, en un unique passage d'entraînement, exactement rectiligne et de vitesse constante $U_t = T^*(X_0) - X_0$, réalisant l'infimum du Théorème 4.1.*

Remarque 9.2. On ne démontre pas ici le Théorème 9.1 (voir Kornilov et al., 2024, pour la preuve complète reposant sur la caractérisation variationnelle des potentiels de Brenier) mais on en indique l'intuition : contraindre T_θ à être un gradient de fonction convexe restreint drastiquement l'espace de recherche à des applications qui *peuvent* être des transports optimaux quadratiques (par le Théorème 4.2, toute application de transport optimal quadratique est de cette forme, et réciproquement). La perte de régression usuelle, minimisée sur cette classe convexe plutôt que sur l'espace de tous les champs mesurables du Théorème 8.1, sélectionne alors directement le potentiel de Brenier, sans étape itérative.

Corollaire 9.1 (Coût d'intégration nul). *Puisque $U_t = T^*(X_0) - X_0$ ne dépend pas de t , l'ODE de génération de la Section 10 s'intègre exactement par un unique pas d'Euler :*

$$\hat{X}_1 = \hat{X}_0 + T_\theta(\hat{X}_0) - \hat{X}_0 = T_\theta(\hat{X}_0).$$

La génération se réduit donc à une unique évaluation du potentiel appris, sans aucune erreur de discrétisation (Section 10).

Méthode	Couplage	Coût entraînement	Pas à l'inférence
CFM (couplage indépendant)	$\pi = p_0 \otimes p_1$	régression, simulation-free	$N \gg 1$ (trajectoires courbes)
OT-CFM (Tong et al., 2023)	OT discret en minibatch, approché	régression + OT par batch	N réduit
OFM (Kornilov et al., 2024)	OT exact (Brenier), sous hypothèses	régression sur potentiel convexe	$N = 1$ exact

10 Génération par intégration numérique

10.1 ODE de génération et erreur de discrétisation

Après entraînement, on remplace le champ exact v_t^π par son approximation $v_\theta(t, x)$. Pour générer, on échantillonne $\hat{X}_0 \sim p_0$ puis on résout

$$\frac{d\hat{X}_t}{dt} = v_\theta(t, \hat{X}_t), \quad \hat{X}_{t=0} \sim p_0.$$

La sortie générée est $\hat{X}_1$.

Définition 10.1 (Schéma d'Euler explicite). Avec $N \in \mathbb{N}^*$ pas et $\Delta t = 1/N$, $t_k = k/N$ pour $k = 0, \dots, N-1$:

$$\hat{X}_{k+1} = \hat{X}_k + \Delta t v_\theta(t_k, \hat{X}_k).$$

Proposition 10.1 (Ordre de convergence d'Euler). Si v_θ est lipschitzienne en x (constante L) uniformément en t , et de classe $\mathcal{C}^1$ en t , l'erreur globale du schéma d'Euler explicite après N pas vérifie

$$\max_{0 \leq k \leq N} \|\hat{X}_k - X_{t_k}\|_2 = O(\Delta t) = O(1/N),$$

c'est-à-dire que le schéma d'Euler est d'ordre 1 (l'erreur locale par pas est en $O(\Delta t^2)$, l'accumulation sur $N = 1/\Delta t$ pas donne une erreur globale en $O(\Delta t)$).

Remarque 10.1. Des schémas d'ordre supérieur (Heun / RK2, en $O(\Delta t^2)$ globalement, ou Runge-Kutta d'ordre 4, en $O(\Delta t^4)$) permettent de réduire fortement le nombre de pas nécessaires pour une précision donnée, au prix d'évaluations supplémentaires de v_θ par pas. Le schéma de Heun s'écrit

$$\tilde{X}_{k+1} = \hat{X}_k + \Delta t v_\theta(t_k, \hat{X}_k), \quad \hat{X}_{k+1} = \tilde{X}_k + \frac{\Delta t}{2} (v_\theta(t_k, \tilde{X}_k) + v_\theta(t_{k+1}, \tilde{X}_{k+1})).$$

Listing 1 – Simulation Euler d'un flow appris

```
@torch.no_grad()
def sample_flow(model, n_samples, n_steps, shape, device):
    x = torch.randn(n_samples, *shape, device=device)
    dt = 1.0 / n_steps

    model.eval()
    for k in range(n_steps):
        t = torch.full((n_samples,), k / n_steps, device=device)
        velocity = model(x, t)
        x = x + dt * velocity

    return x
```

Dans ce code, l'appel `model(x, t)` correspond mathématiquement à $v_\theta(t, x)$; l'ordre des arguments est seulement une convention d'implémentation.

Listing 2 – Schéma de Heun (ordre 2)

```
@torch.no_grad()
def sample_flow_heun(model, n_samples, n_steps, shape, device):
    x = torch.randn(n_samples, *shape, device=device)
    dt = 1.0 / n_steps

    model.eval()
    for k in range(n_steps):
```

```

t0 = torch.full((n_samples,), k / n_steps, device=device)
t1 = torch.full((n_samples,), (k + 1) / n_steps, device=device)

v0 = model(x, t0)
x_pred = x + dt * v0
v1 = model(x_pred, t1)

x = x + 0.5 * dt * (v0 + v1)

return x

```

11 Exemple 1 : distribution 2D make_moons

11.1 Construction du dataset

On veut apprendre à transformer un bruit gaussien 2D en une distribution en forme de deux lunes :

$$X_0 \sim \mathcal{N}(0, I_2), \quad X_1 \sim p_{\text{moons}},$$

où p_{moons} désigne la loi empirique des points du dataset `make_moons`, avec couplage indépendant $\pi = p_0 \otimes p_{\text{moons}}$. On utilise le chemin cosinus (§ 7.2) :

$$X_t = \cos\left(\frac{\pi}{2}t\right)X_0 + \sin\left(\frac{\pi}{2}t\right)X_1, \quad U_t = -\frac{\pi}{2}\sin\left(\frac{\pi}{2}t\right)X_0 + \frac{\pi}{2}\cos\left(\frac{\pi}{2}t\right)X_1.$$

11.2 Code minimal

Listing 3 – Interpolation cosinus et cible de vitesse

```

def cosine_schedule_torch(t):
    c = torch.pi / 2.0
    alpha = torch.cos(c * t)
    beta = torch.sin(c * t)
    d_alpha = -c * torch.sin(c * t)
    d_beta = c * torch.cos(c * t)
    return alpha, beta, d_alpha, d_beta

x0 = torch.randn(n_train, 2, device=device)
x1 = moons_torch[torch.randint(0, len(moons_torch), (n_train,), device=device)]
t = torch.rand(n_train, 1, device=device)

alpha, beta, d_alpha, d_beta = cosine_schedule_torch(t)
xt = alpha * x0 + beta * x1
target_velocity = d_alpha * x0 + d_beta * x1

```

Le modèle est un MLP recevant les features $(x_t, t, \sin(\pi t), \cos(\pi t))$, où $x_t \in \mathbb{R}^2$ est une réalisation de X_t ; le modèle prédit une vitesse dans $\mathbb{R}^2$, cible de la régression du Théorème 8.1.

Listing 4 – Loss de flow matching pour l'exemple 2D

```

features = model_features_torch(xt, t)
pred_velocity = flow_model(features)
loss = F.mse_loss(pred_velocity, target_velocity)

```

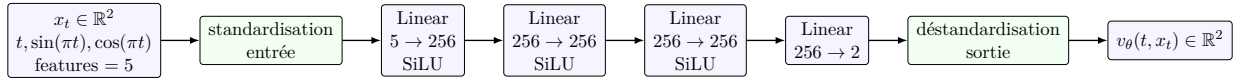


FIGURE 2 – Schéma du MLP utilisé dans le notebook pour l’expérience `make_moons`. Le réseau prédit une vitesse 2D à partir de la position courante et d’un encodage temporel trigonométrique.

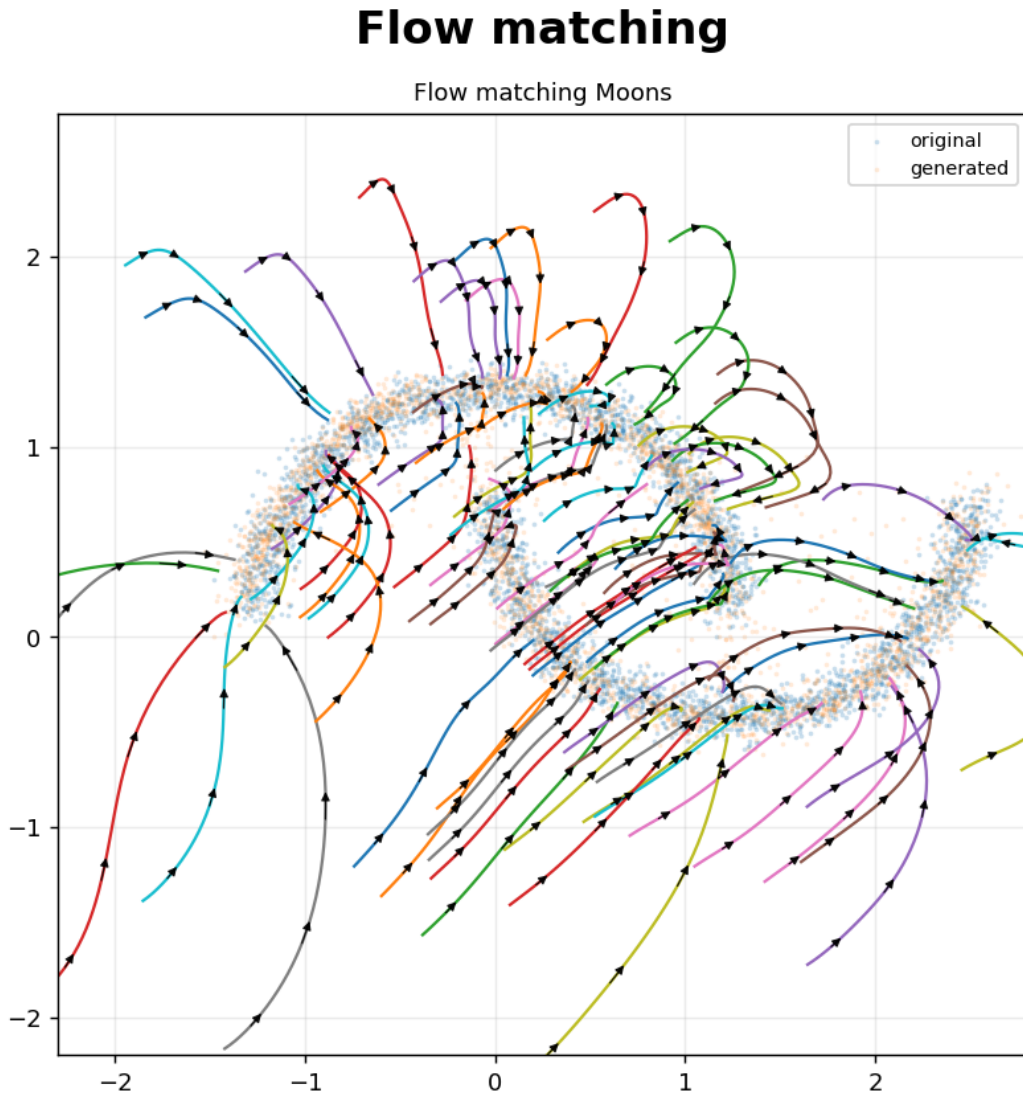


FIGURE 3 – Figure extraite du notebook : distribution `make_moons`, échantillons générés et trajectoires obtenues en intégrant le champ de vitesse appris par Euler.

11.3 Schéma du modèle et sortie du notebook

11.4 Variante OT-CFM sur `make_moons` : comparaison des vecteurs

Dans l'expérience précédente, le couplage utilisé est indépendant :

$$(X_0, X_1) \sim p_0 \otimes p_{\text{moons}}.$$

Les paires bruit-donnée sont donc arbitraires. Cela peut produire des vecteurs conditionnels longs et désordonnés, notamment pour le chemin linéaire $U_t = X_1 - X_0$.

La variante OT-CFM remplace ce couplage indépendant par un couplage optimal *dans chaque minibatch*. Si le batch source est $\{x_0^{(i)}\}_{i=1}^m$ et le batch cible est $\{x_1^{(j)}\}_{j=1}^m$, on construit la matrice de coûts

$$C_{ij} = \|x_0^{(i)} - x_1^{(j)}\|_2^2.$$

On cherche ensuite la permutation optimale

$$\sigma^* = \arg \min_{\sigma \in \mathfrak{S}_m} \sum_{i=1}^m C_{i, \sigma(i)}.$$

Le batch d'entraînement devient alors

$$(x_0^{(i)}, x_1^{(\sigma^*(i))})_{i=1}^m.$$

Pour le schedule linéaire, les vitesses cibles comparées sont donc

$$u_{\text{ind}}^{(i)} = x_{1, \text{ind}}^{(i)} - x_0^{(i)}, \quad u_{\text{OT}}^{(i)} = x_1^{(\sigma^*(i))} - x_0^{(i)}.$$

Listing 5 – Matching OT minibatch pour `make_moons`

```
from scipy.optimize import linear_sum_assignment

def minibatch_ot_pairs(x0, x1):
    # x0: (m, d), x1: (m, d)
    cost = ((x0[:, None, :] - x1[None, :, :]) ** 2).sum(axis=-1)
    row_ind, col_ind = linear_sum_assignment(cost)
    x1_ot = x1[col_ind]
    return x0[row_ind], x1_ot

x0_ot, x1_ot = minibatch_ot_pairs(x0_batch, x1_batch)
target_velocity_ot = x1_ot - x0_ot
```

12 Exemple 2 : MNIST avec U-Net

12.1 Passage aux images

Pour MNIST, chaque image est un tenseur $x_1 \in \mathbb{R}^{1 \times 28 \times 28}$, réalisation de $X_1 \sim p_{\text{data}}$. On choisit un bruit $X_0 \sim \mathcal{N}(0, I_d)$, $d = 1 \cdot 28 \cdot 28$, de même taille que l'image. Le réseau doit prédire une vitesse image $v_\theta(t, x_t) \in \mathbb{R}^{1 \times 28 \times 28}$. Comme la sortie a la même taille que l'entrée, un U-Net est naturel : il combine des features locales et globales, puis reconstruit une image de vitesse, en approximant $v_t^\pi(x) = \mathbb{E}[U_t \mid X_t = x]$ (Théorèmes 6.1 et 8.1).

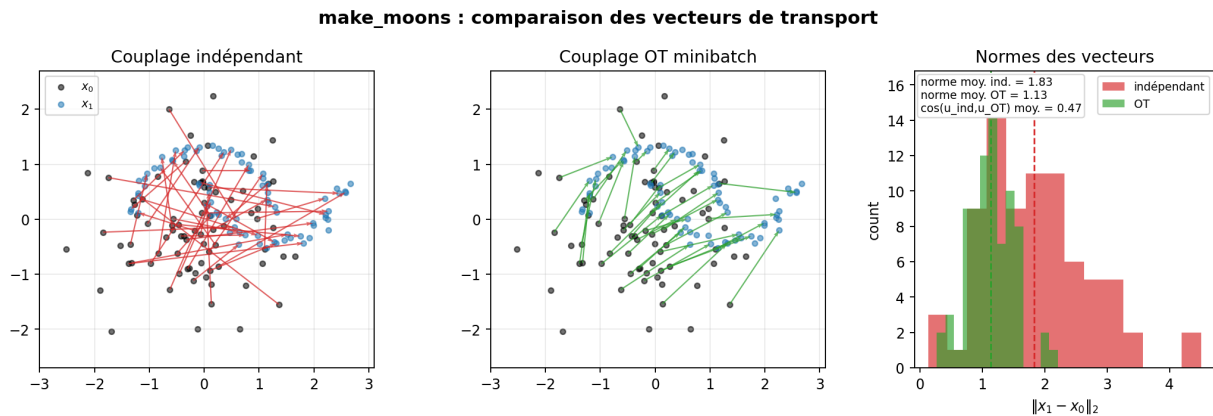


FIGURE 4 – Comparaison pratique sur `make_moons`. À gauche, le couplage indépendant produit des vecteurs plus longs et plus croisés. Au centre, le couplage OT en minibatch associe chaque bruit à une cible plus proche au sens quadratique. À droite, les normes $\|x_1 - x_0\|_2$ montrent que l'OT réduit l'énergie cinétique moyenne du transport.

12.2 Interpolation utilisée

Listing 6 – Interpolation linéaire ou cosinus pour MNIST

```
def interpolate_flow(x0, x1, t, interpolation="linear"):
    t_view = t.view(-1, 1, 1, 1)

    if interpolation == "linear":
        xt = (1 - t_view) * x0 + t_view * x1
        target_velocity = x1 - x0

    elif interpolation == "cos":
        angle = torch.pi * t_view / 2
        alpha = torch.cos(angle)
        beta = torch.sin(angle)
        d_alpha = -(torch.pi / 2) * torch.sin(angle)
        d_beta = (torch.pi / 2) * torch.cos(angle)

        xt = alpha * x0 + beta * x1
        target_velocity = d_alpha * x0 + d_beta * x1

    return xt, target_velocity
```

12.3 Boucle d'entraînement

Listing 7 – Étape d'entraînement flow matching sur MNIST

```
for x1, _ in train_loader:
    x1 = x1.to(device)
    x1 = x1 * 2 - 1

    x0 = torch.randn_like(x1)
    t = torch.rand(x1.shape[0], device=device)

    xt, target_velocity = interpolate_flow(x0, x1, t, interpolation="cos")
    pred_velocity = model(xt, t)

    loss = F.mse_loss(pred_velocity, target_velocity)
```

```
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()
```

12.4 Schéma du U-Net et sortie du notebook

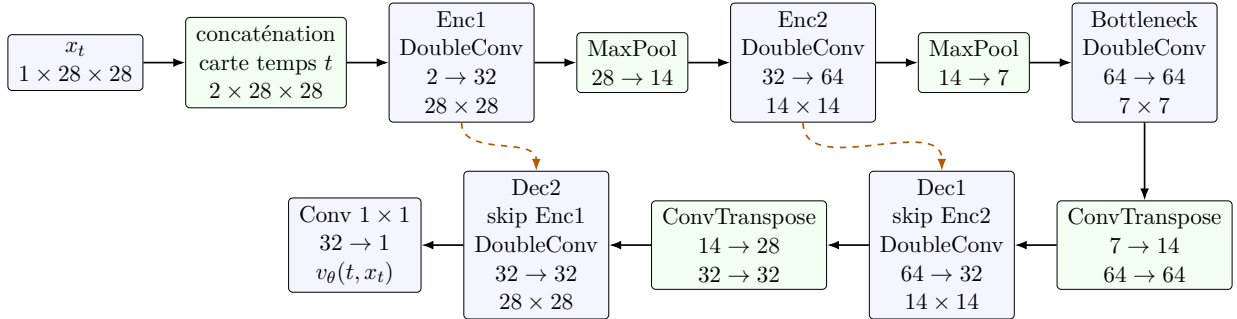


FIGURE 5 – Schéma du U-Net utilisé dans le notebook. Chaque bloc `DoubleConv` correspond à deux convolutions 3×3 suivies de ReLU. Le temps t est injecté comme une carte constante concaténée à l'image bruitée x_t .

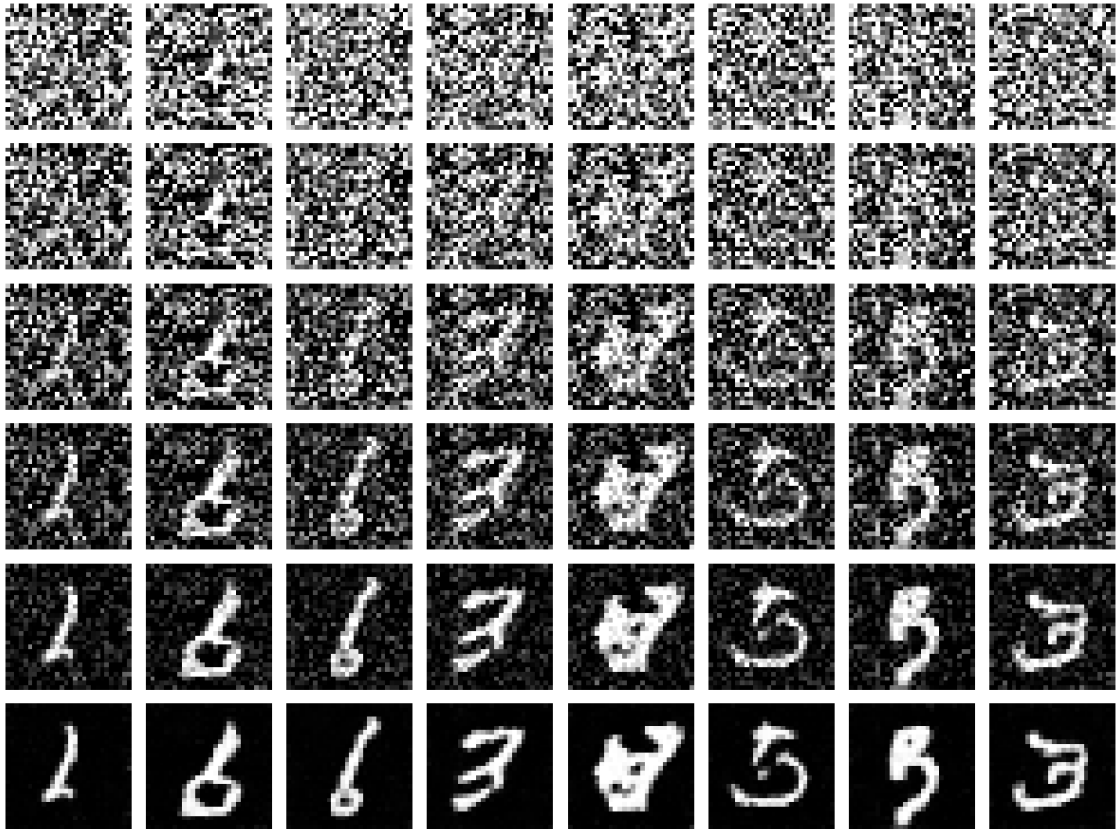


FIGURE 6 – Figure extraite du notebook : évolution de plusieurs échantillons MNIST pendant l'intégration de l'ODE apprise. Les lignes correspondent à des instants de simulation successifs.

12.5 Variante OT-CFM sur MNIST : comparaison des vecteurs en espace image

Pour les images, le même principe s'applique après vectorisation :

$$x_0^{(i)}, x_1^{(j)} \in \mathbb{R}^{784}, \quad C_{ij} = \|x_0^{(i)} - x_1^{(j)}\|_2^2.$$

On résout à chaque batch un problème d'affectation optimal entre les bruits et les images du batch. Le vecteur de transport est alors une image de même taille que MNIST :

$$u^{(i)} = x_1^{(\sigma^*(i))} - x_0^{(i)} \in \mathbb{R}^{1 \times 28 \times 28}.$$

On peut visualiser ce vecteur pixel par pixel avec une colormap signée : les zones rouges correspondent aux pixels que le champ doit augmenter, les zones bleues aux pixels qu'il doit diminuer.

Listing 8 – OT-CFM en espace pixel pour MNIST

```
from scipy.optimize import linear_sum_assignment

def minibatch_ot_images(x0, x1):
    # x0, x1: (m, 1, 28, 28), valeurs dans [-1, 1]
    m = x0.shape[0]
    x0_flat = x0.reshape(m, -1)
    x1_flat = x1.reshape(m, -1)
    cost = ((x0_flat[:, None, :] - x1_flat[None, :, :]) ** 2).sum(axis=-1)
    row_ind, col_ind = linear_sum_assignment(cost)
    return x0[row_ind], x1[col_ind]

x0_ot, x1_ot = minibatch_ot_images(x0_batch, x1_batch)
xt, target_velocity = interpolate_flow(x0_ot, x1_ot, t, interpolation="cos")
```

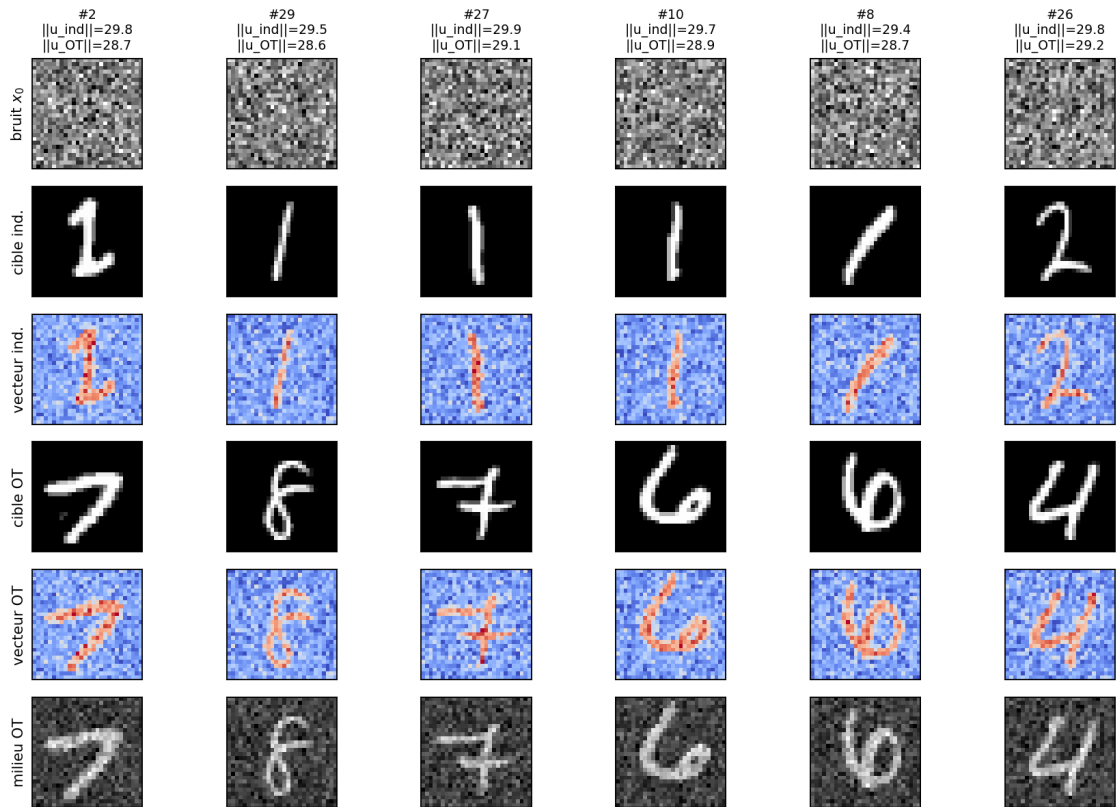
Point d'attention

Pour MNIST, cette procédure réalise un transport optimal dans l'espace pixel $\mathbb{R}^{784}$. Elle minimise bien une distance quadratique entre tenseurs, mais ce n'est pas nécessairement une distance sémantique entre chiffres. Deux images visuellement proches pour un humain peuvent être éloignées en norme pixel, et réciproquement. Pour améliorer ce point, on peut calculer l'OT dans un espace de features appris plutôt que directement dans l'espace image.

13 Comparaison expérimentale : CFM contre OT-CFM

Les sections précédentes ont entraîné, sur `make_moons` et sur MNIST, un flow matching classique (CFM, couplage indépendant $\pi = p_0 \otimes p_1$) et comparé *ponctuellement* les vecteurs de transport obtenus avec un couplage OT en minibatch (Figures 4 et 7). Cette section va plus loin : on entraîne un *second* modèle complet avec la perte OT-CFM (couplage optimal en minibatch à chaque itération, Section 9) sur les deux mêmes jeux de données, et on compare les deux modèles entraînés – pas seulement les vecteurs de couplage – sur la rectitude des trajectoires, l'énergie cinétique et la qualité de génération en fonction du nombre de pas d'intégration. Le code complet se trouve dans `ot_flow_matching.ipynb`.

MNIST : comparaison des vecteurs de transport en espace pixel



Batch size = 32 | norme moyenne indépendante = 29.1 | norme moyenne OT = 28.8 | cosinus moyen = 0.78

FIGURE 7 – Comparaison MNIST en espace pixel. Chaque colonne fixe un bruit x_0 . Les lignes comparent la cible issue d'un couplage indépendant, le vecteur indépendant, la cible issue du matching OT, le vecteur OT, puis le milieu de trajectoire OT. Les normes affichées montrent que l'OT réduit en général la longueur du vecteur de transport, mais la correspondance reste définie par une distance pixel.

Point d'attention

Pour isoler l'effet du couplage (et non celui de la forme du chemin conditionnel), les deux modèles de cette section utilisent le **schedule linéaire** $X_t = (1-t)X_0 + tX_1$, $U_t = X_1 - X_0$ (§ 7.1), et non le schedule cosinus utilisé dans `notebook.ipynb`. C'est uniquement avec un chemin conditionnel déjà rectiligne que le couplage OT peut produire des trajectoires *marginales* rectilignes (Théorème 4.2 et remarque suivante) : avec un chemin courbe, comparer les deux couplages mélangerait deux effets géométriques distincts et rendrait la comparaison ininterprétable.

13.1 Rectitude des trajectoires et énergie cinétique sur `make_moons`

Les deux modèles (MLP identique, § 10.2) sont entraînés 4000 itérations chacun, puis on simule 600 trajectoires par intégration d'Euler à $N = 100$ pas, à partir des *mêmes* points de bruit X_0 pour les deux modèles.

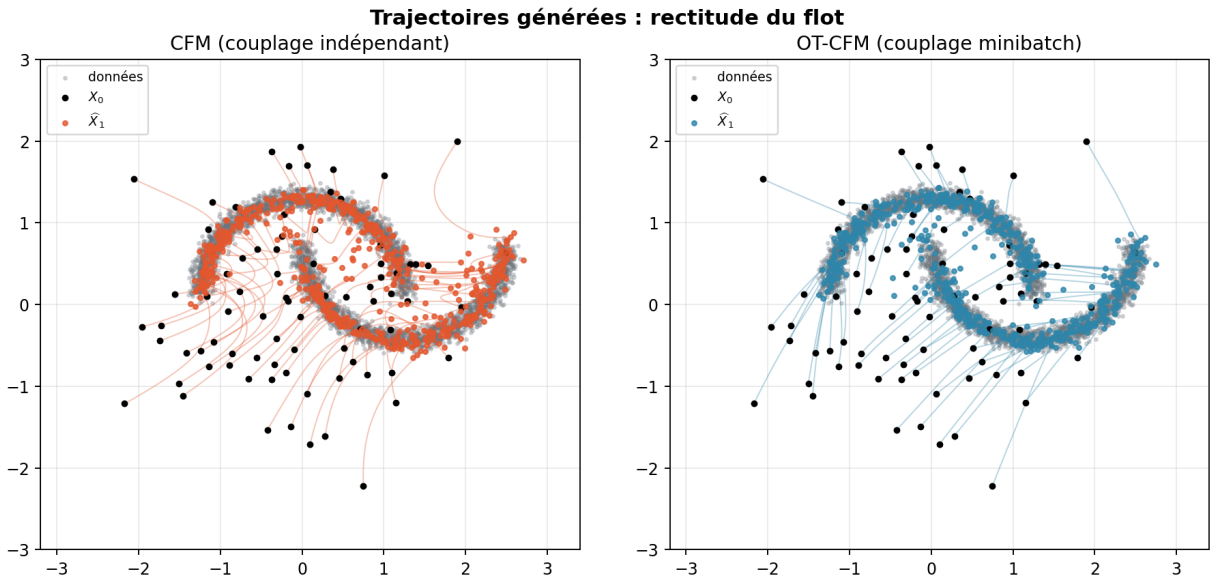


FIGURE 8 – Trajectoires générées par le modèle CFM (gauche) et le modèle OT-CFM (droite), à partir des mêmes échantillons de bruit. Les trajectoires OT-CFM sont visiblement quasi rectilignes, contrairement aux trajectoires CFM qui contournent la masse de probabilité.

Pour quantifier cette différence, on mesure, pour chaque trajectoire, l'écart quadratique moyen au segment $[\hat{X}_0, \hat{X}_1]$ (« rectitude »), et l'énergie cinétique moyenne $\int_0^1 \mathbb{E}[\|v_\theta(t, X_t)\|_2^2] dt$ (approximation par somme de Riemann le long des trajectoires simulées, cf. Théorème 4.1).

Sur cette expérience, l'erreur de rectitude moyenne passe de 0,251 (CFM) à 0,013 (OT-CFM), et l'énergie cinétique moyenne de 1,80 à 1,02, soit une réduction de 43% – cohérent avec le Théorème 4.1 : le couplage optimal en minibatch rapproche l'énergie cinétique du transport de la borne inférieure donnée par $W_2(p_0, p_1)^2$.

13.2 Qualité de génération en fonction du nombre de pas (NFE)

Une conséquence pratique directe de la rectitude : moins de pas d'intégration suffisent pour approcher la distribution cible. On mesure la distance de Wasserstein empirique $\hat{W}_2$ (coût quadratique, calculée exactement entre deux échantillons de même taille par affectation optimale, comme au § 9) entre les échantillons générés et `make_moons`, pour un nombre de pas N croissant.

À $N = 1$ pas (un unique pas d'Euler, c'est-à-dire $\hat{X}_1 = \hat{X}_0 + v_\theta(0, \hat{X}_0)$), le modèle OT-CFM atteint déjà $\hat{W}_2 \approx 0,17$, contre $\hat{W}_2 \approx 1,13$ pour le modèle CFM : à couplage optimal, une trajectoire

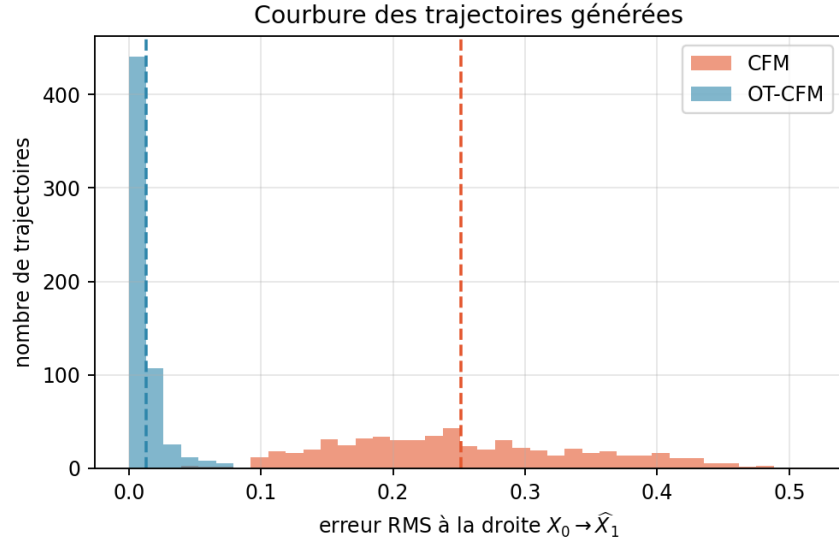


FIGURE 9 – Distribution de l’erreur de rectitude (écart RMS à la droite $\hat{X}_0 \rightarrow \hat{X}_1$) pour les 600 trajectoires générées par chaque modèle.

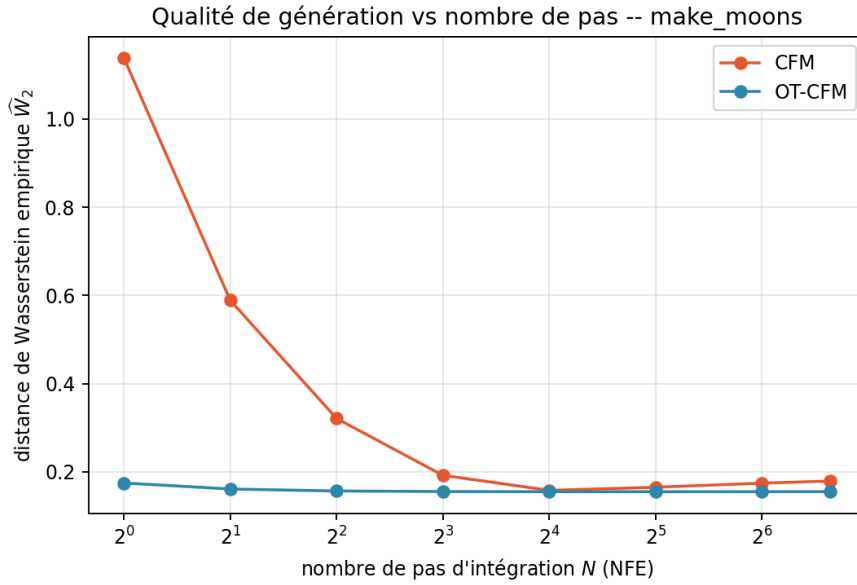


FIGURE 10 – Distance de Wasserstein empirique $\hat{W}_2$ entre échantillons générés et données, en fonction du nombre de pas d’intégration N (échelle log). Le modèle OT-CFM atteint sa qualité asymptotique dès $N = 1$ à 2 pas, tandis que le modèle CFM nécessite $N \gtrsim 16$ pas pour converger au même niveau.

quasi rectiligne se prête bien à une intégration en très peu de pas, alors qu’une trajectoire courbe nécessite de nombreux petits pas pour suivre correctement sa courbure (erreur d’Euler en $O(\Delta t)$, Section 10).

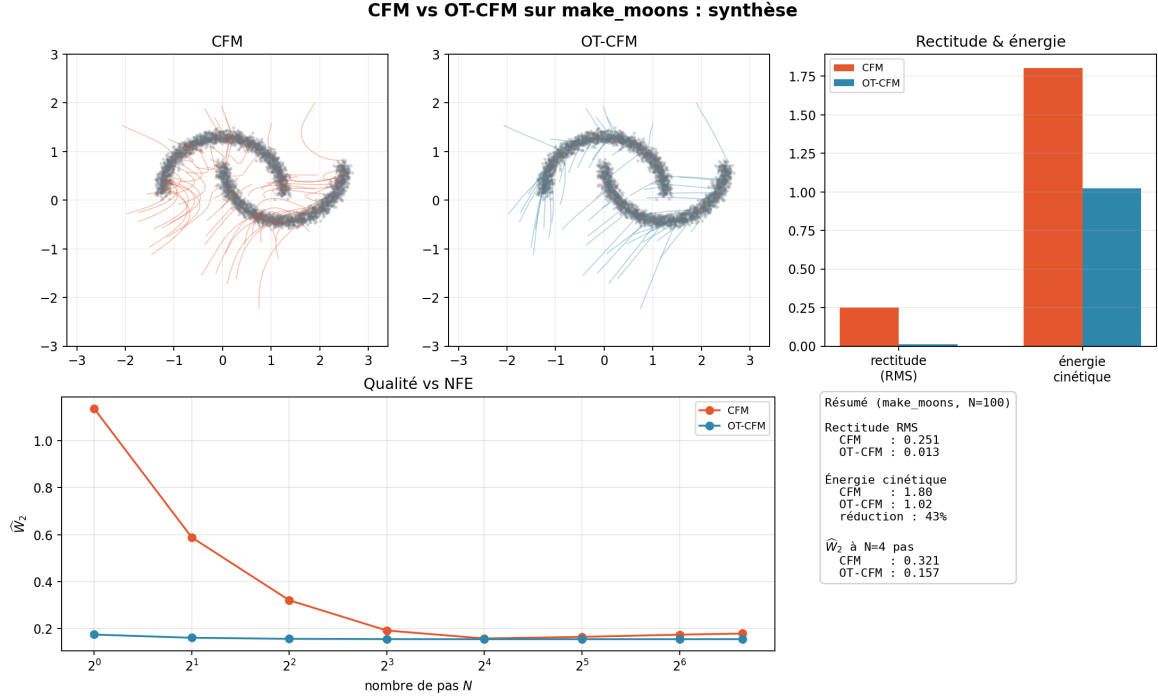


FIGURE 11 – Synthèse de la comparaison sur `make_moons` : trajectoires, rectitude, énergie cinétique et qualité en fonction du NFE.

13.3 Sur MNIST : qualité visuelle à faible NFE

On répète l’expérience avec le U-Net (§ 12) entraîné sur MNIST, en couplant l’OT en minibatch directement dans l’espace pixel $\mathbb{R}^{784}$ (§ 9, comme à la Figure 7).

Point d’attention

Sur MNIST, l’écart de qualité entre CFM et OT-CFM à faible NFE est nettement moins marqué que sur `make_moons` : dès $N = 2$ pas, les deux modèles produisent des chiffres reconnaissables de qualité comparable. Deux raisons principales : d’une part, l’espace $\mathbb{R}^{784}$ est haute dimension et le couplage OT en minibatch (taille 128) n’y approche que très grossièrement le couplage optimal continu du Théorème 4.2 (Proposition 9.1 requiert $m \rightarrow \infty$) ; d’autre part, la distance quadratique en espace pixel n’est pas nécessairement celle qui gouverne la difficulté perceptuelle de la génération. Le gain de rectitude de l’OT-CFM est donc plus net et plus facilement mesurable en basse dimension (`make_moons`) que sur des images brutes en pixels.

14 Résumé conceptuel

1. Un flot Φ_t transporte p_0 vers p_1 au sens de l’équation de continuité (EC) ; c’est l’objet mathématique commun à toutes les méthodes de ce cours.
2. La méthode *classique* (CNF, Chen *et al.*, 2018) entraîne ce flot par maximum de vraisemblance exacte, via la formule de changement de variables instantanée (Proposition 5.1) ; elle est

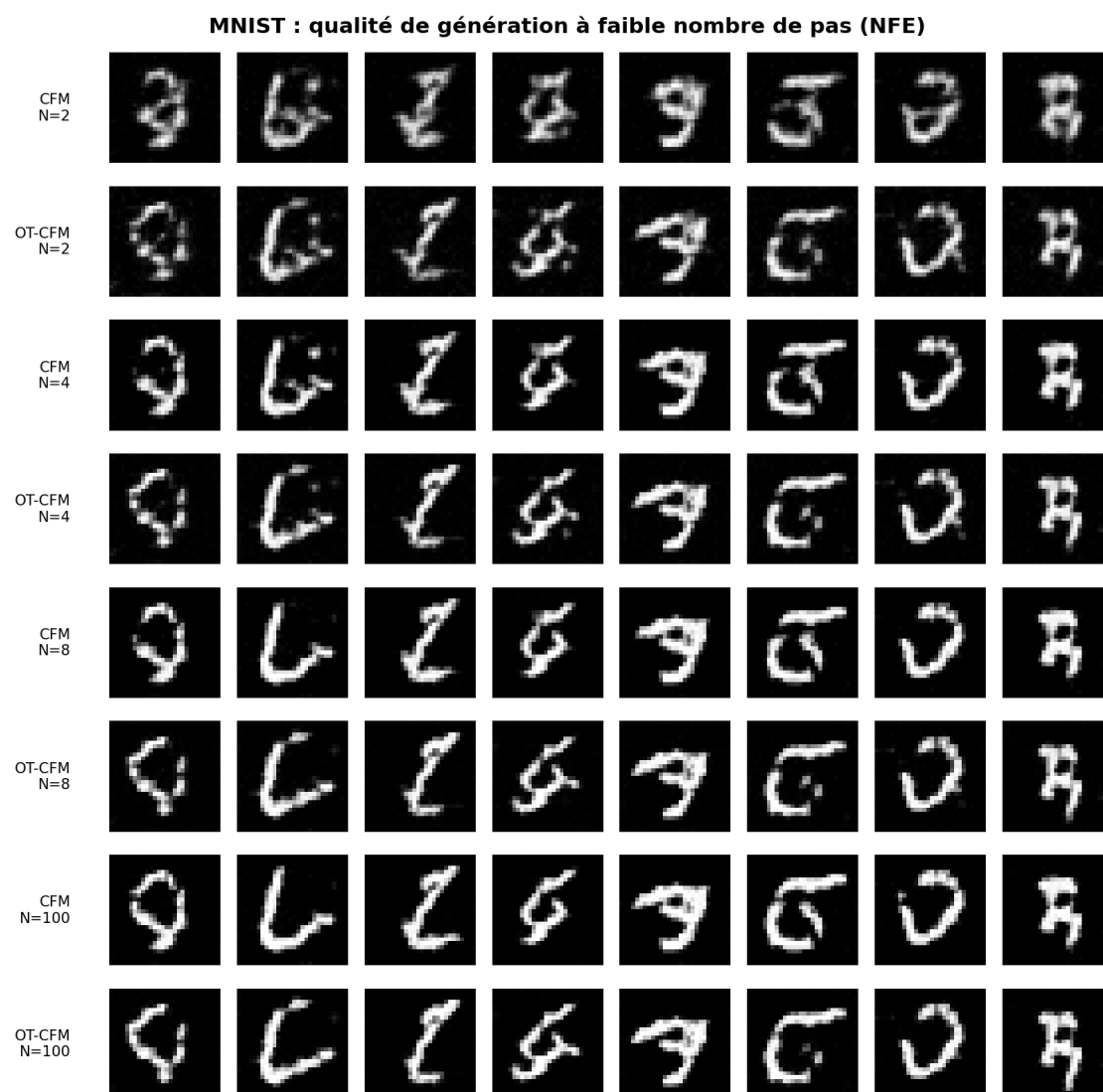


FIGURE 12 – Échantillons MNIST générés par les modèles CFM et OT-CFM entraînés, à nombre de pas N croissant (2, 4, 8, 100), à partir des mêmes bruits initiaux.

simulation-based : coûteuse (ODE avant et arrière, trace de jacobien, méthode adjointe) et sans lien direct avec l’optimalité du transport.

3. Le flow matching choisit un couplage $\pi \in \Pi(p_0, p_{\text{data}})$ et un chemin de probabilité conditionnel affine $X_t = \alpha(t)X_0 + \beta(t)X_1$, de vitesse exacte $U_t = \dot{\alpha}(t)X_0 + \dot{\beta}(t)X_1$, calculable en forme close.
4. Le champ marginal $v_t^\pi(x) = \mathbb{E}[U_t \mid X_t = x]$ transporte exactement p_t (Théorème 6.1, par désintégration de π), et c’est aussi l’unique minimiseur L^2 de la perte de régression $\mathcal{L}_{\text{CFM}}$ (Théorème 8.1) : entraîner v_θ par MSE sur des cibles conditionnelles suffit donc, sans jamais intégrer d’ODE pendant l’entraînement – c’est le gain *simulation-free* par rapport au CNF.
5. Le couplage indépendant $\pi = p_0 \otimes p_1$, utilisé par défaut, n’atteint pas l’énergie cinétique minimale du Théorème 4.1 : les couplages OT en minibatch (OT-CFM, Tong *et al.*, 2023) s’en rapprochent, et la paramétrisation convexe (Optimal Flow Matching, Kornilov *et al.*, 2024) l’atteint exactement grâce au théorème de Brenier (Théorème 4.2), au prix d’une classe de champs plus restreinte.
6. Pour générer, on part d’un bruit $\hat{X}_0 \sim p_0$ et l’on intègre numériquement l’ODE apprise (Euler, Heun, RK4), avec une erreur globale contrôlée par l’ordre du schéma et le pas Δt – sauf pour l’OFM, où un unique pas suffit exactement.
7. Expérimentalement (Section 13), le couplage OT en minibatch réduit bien l’énergie cinétique et rend les trajectoires quasi rectilignes sur `make_moons`, ce qui permet de générer en très peu de pas ($N = 1$ à 2) une qualité que le couplage indépendant n’atteint qu’à $N \gtrsim 16$ pas ; l’effet est en revanche beaucoup plus ténu sur MNIST, où l’OT en minibatch, à taille de batch finie, n’approche que grossièrement le couplage optimal continu en haute dimension.

15 Exercices

Exercice 1 — Conditions aux bords et régularité

Vérifier que le schedule $\alpha(t) = \cos(\frac{\pi}{2}t)$, $\beta(t) = \sin(\frac{\pi}{2}t)$ satisfait $X_{t=0} = X_0$, $X_{t=1} = X_1$, et montrer que (α, β) est de classe $\mathcal{C}^\infty$ sur $[0, 1]$.

Exercice 2 — Vitesse linéaire

Pour $X_t = (1 - t)X_0 + tX_1$, montrer que $U_t = \frac{dX_t}{dt}$ ne dépend pas de t , et calculer l’énergie cinétique moyenne $\int_0^1 \mathbb{E}[\|U_t\|_2^2] dt$ en fonction de $\mathbb{E}[\|X_1 - X_0\|_2^2]$.

Exercice 3 — Variance du chemin

Supposons X_0, X_1 indépendantes, centrées, de covariance I_d . Calculer $\text{Cov}(X_t)$ pour le schedule linéaire, puis pour le schedule cosinus. En déduire pour quelle valeur de t l’écart entre les deux schedules est maximal.

Exercice 4 — Démonstration du théorème de marginalisation en dimension 1

Traiter explicitement le cas $d = 1$, $\pi = p_0 \otimes p_1$ (couplage indépendant), $X_t = (1 - t)X_0 + tX_1$. Écrire p_t comme un produit de convolution de lois renormalisées, puis retrouver $v_t^\pi(x) = \mathbb{E}[X_1 - X_0 \mid X_t = x]$ par un calcul direct sur les densités, sans invoquer la désintégration abstraite.

Exercice 5 — Ordre du schéma d’Euler

En utilisant un développement de Taylor de la solution exacte $X_{t_{k+1}}$ autour de t_k , montrer que l’erreur locale du schéma d’Euler explicite est en $O(\Delta t^2)$, puis que l’erreur globale après $N = 1/\Delta t$ pas est en $O(\Delta t)$.

Exercice 6 — Mise en pratique

Implémenter le schéma d'Euler et le schéma de Heun de la Section 10, comparer visuellement (nombre de pas $N = 20, 100, 500$) la qualité des échantillons générés sur `make_moons`, et estimer empiriquement l'ordre de convergence en mesurant une distance de Wasserstein empirique entre $\hat{X}_1$ et p_{moons} en fonction de N .

Exercice 7 — Changement de variables instantané

En dimension $d = 1$, pour $v_t(x) = a(t)x$ (champ linéaire), résoudre explicitement (ODE), en déduire $\Phi_t(x_0)$, puis vérifier directement, par un calcul de densité gaussienne, la formule de la Proposition 5.1 lorsque $p_0 = \mathcal{N}(0, 1)$.

Exercice 8 — Couplage optimal en dimension 1

En dimension $d = 1$, montrer que le couplage optimal du Théorème 4.2 pour le coût quadratique est donné par $T^* = F_1^{-1} \circ F_0$, où F_0, F_1 sont les fonctions de répartition de p_0, p_1 (couplage monotone). Comparer, sur un petit exemple numérique à deux composantes gaussiennes pour p_1 , la trajectoire donnée par ce couplage à celle du couplage indépendant, et calculer l'écart d'énergie cinétique prédit par le Théorème 4.1.

16 Ouverture

Le flow matching présenté ici repose sur des chemins conditionnels affines entre bruit et données, affinés par des couplages de transport optimal (Section 9). Plusieurs extensions actives en recherche méritent encore d'être mentionnées :

- **Rectified flow** (Liu *et al.*, 2022) : on « redresse » itérativement les trajectoires en ré-entraînant le modèle sur des paires $(\hat{X}_0, \hat{X}_1)$ générées par le modèle précédent, ce qui rapproche le couplage effectif du couplage optimal de Monge (Théorème 4.1) sans toutefois garantir l'exactitude en un nombre fini d'itérations, contrairement à l'Optimal Flow Matching (§ 9.2).
- **Interpolants stochastiques** (Albergo–Vanden-Eijnden, 2023) : cadre unifiant flow matching déterministe et diffusion stochastique, en ajoutant un terme brownien contrôlé au chemin conditionnel tout en conservant les mêmes marginales.
- **Guidance sans classifieur** (*classifier-free guidance*) : extrapolation entre un champ conditionnel $v_\theta(t, x | c)$ et un champ non conditionnel $v_\theta(t, x)$ pour renforcer l'adéquation à une condition c (classe, texte, etc.) au moment de l'échantillonnage.

Mais le cœur de la méthode reste celui établi dans ce cours : après avoir vu pourquoi la méthode classique des CNF, coûteuse en simulation, a cédé la place au flow matching (Section 5), puis comment ce dernier peut lui-même être rapproché de l'optimum du transport optimal (Section 9), retenons l'idée directrice : apprendre, par simple régression L^2 sur des vitesses conditionnelles explicites, un champ de vitesse marginal qui transporte rigoureusement une distribution simple vers la distribution des données.

Références

- R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud (2018), *Neural Ordinary Differential Equations*, NeurIPS. (Continuous Normalizing Flows, méthode adjointe.)
- W. Grathwohl, R. T. Q. Chen, J. Bettencourt, I. Sutskever, D. Duvenaud (2018), *FFJORD : Free-form Continuous Dynamics for Scalable Reversible Generative Models*, arXiv :1810.01367.
- J.-D. Benamou, Y. Brenier (2000), *A computational fluid mechanics solution to the Monge-Kantorovich mass transfer problem*, Numerische Mathematik.

- Y. Brenier (1991), *Polar factorization and monotone rearrangement of vector-valued functions*, Communications on Pure and Applied Mathematics.
- Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le (2023), *Flow Matching for Generative Modeling*, arXiv :2210.02747.
- A. Tong, N. Malkin, G. Huguet, Y. Zhang, K. Fatras, J. Rector-Brooks, G. Wolf, Y. Bengio (2023), *Improving and generalizing flow-based generative models with minibatch optimal transport*, arXiv :2302.00482.
- N. Kornilov, P. Mokrov, A. Gasnikov, A. Korotin (2024), *Optimal Flow Matching : Learning Straight Trajectories in Just One Step*, arXiv :2403.13117.
- X. Liu, C. Gong, Q. Liu (2022), *Flow Straight and Fast : Learning to Generate and Transfer Data with Rectified Flow*, arXiv :2209.03003.
- M. S. Albergo, E. Vanden-Eijnden (2023), *Building Normalizing Flows with Stochastic Interpolants*, arXiv :2209.15571.
- B. Amos, L. Xu, J. Z. Kolter (2017), *Input Convex Neural Networks*, arXiv :1609.07152.
- C. Villani (2003), *Topics in Optimal Transportation*, American Mathematical Society.